Fakultät Informatik an der Technischen Hochschule Nürnberg



Bachelorarbeit

My very fancy thesis title

Georg Simon Ohm
Matrikelnummer: 1234567

Prüfer: Prof. Dr. Max Mustermann
Prof. Dr. Monika Musterfrau

Betreuer: Prof. Dr. Max Musterbetreuer
Prof. Dr. Monika Musterbetreuerin
Example AG

© Example AG 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von Example AG ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Danksagung

Only if you want to thank someone.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens eine halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird. Das Template warnt vor falschen Angaben.

Keywords
Container Image, Software Supply Chain Security

Abstract

Only if thesis is written in English.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation und Relevanz ... 1
1.2 Problemstellung ... 1
1.3 Fallstudie und Zielsetzung ... 2
1.4 Forschungsfrage ... 2
1.5 Aufbau der Arbeit ... 2

2 Theoretischer Hintergrund **4**

3 Methodisches Vorgehen **6**
3.1 Forschungsansatz und Erkenntnisziel ... 6
3.2 Untersuchungsgegenstand und Analyseverfahren ... 6
3.3 Ablauf der Entwicklung ... 7
3.3.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung ... 7
3.3.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen ... 7
3.3.3 Phase 3: Konsolidierte Prüfung und Härtung ... 7
3.4 Ableitung des Leitfadens und Bewertung ... 8

4 Analyse und Anforderungsdefinition **9**
4.1 Vom übergeordneten Ziel zur Fallauswahl ... 9
4.2 Funktionen und Architektur des Ausgangsartefakts ... 10
4.2.1 Funktionaler Untersuchungsumfang ... 10
4.2.2 Technische Komponenten und Repository-Struktur ... 10
4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen ... 13
4.3 Zielarchitektur und Sicherheitsbegriff ... 14
4.4 Anforderungskatalog ... 15
4.4.1 A1: Lauffähige Desktop-Anwendung ... 15
4.4.2 A2: Dreischichtenarchitektur ... 15
4.4.3 A3: Begrenzung des Refactoring-Aufwands ... 15
4.4.4 A4: Lokale Kommunikation ohne HTTP-Wrapper ... 15
4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen ... 15
4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“ ... 16
4.4.7 A7: Härtung gemäß Electron Security Guidelines ... 16
4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit ... 16
4.4.9 A9: Nachvollziehbarer Migrationsleitfaden ... 16
4.5 Abgrenzung des Untersuchungsumfangs ... 16

5 Entwurf, Migration und Migrationsleitfaden **18**
5.1 Struktur des Migrationsleitfadens ... 18

5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung 19
5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule 19
5.2.2 Anpassung der Tabellendefinitionen an SQLite 20
5.2.3 Anpassung der Datenbankabhängigkeiten 21
5.2.4 Erzeugung versionierter Schemamigrationen 22
5.2.5 Datenbankinitialisierung beim Anwendungsstart 22
5.2.6 Übernahme des API-Vertrags und Anpassung der Codegenerierung 23
5.2.7 Einrichtung des IPC-Transports und der Preload-Schnittstelle 24
5.2.8 Validierung und Zuordnung der Operationen im Main-Prozess 25
5.2.9 Begründung und Grenzen der vertragsbasierten Generierung 25
5.2.10 Übernahme der React-Oberfläche in den Renderer 27
5.2.11 Abgleich von Build-Konfiguration und Frontend-Abhängigkeiten 28
5.2.12 Anbindung der übernommenen Seiten und Anpassung der Datumswerte 29
5.2.13 Anpassung der Navigation an lokale Renderer-Ressourcen 29
5.2.14 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad 29
5.2.15 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung 30
5.2.16 Wiederkehrender Verarbeitungspfad und fachliche Prüfung 31
5.2.17 Migration des iCal-Dateiimports 32
5.2.18 Ergebnisbehandlung und Prüfung des Dateiimports 33
5.2.19 CSV als ergänzende Anwendung des Importmusters 33
5.2.20 Einordnung der lokalen Funktionsmigration 33
5.3 Phase 2: Von der Offline-Kernanwendung zur Local-First-Kollaboration 33
5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung 34
5.5 Ausarbeitung der einzelnen Migrationsschritte 35

6 Evaluation und Demonstration **36**

7 Diskussion **39**

8 Fazit und Ausblick **41**

A Ursprüngliche Erstellungs-Prompts **49**
A.1 Prompt 1: Persönliche Studienorganisation 49
A.2 Prompt 2: Studiengruppen und Zusammenarbeit 51

Abkürzungsverzeichnis

Kapitel 1

Einleitung

1.1 Motivation und Relevanz

Viele Menschen möchten Anwendungen für den persönlichen Gebrauch erstellen, verfügen jedoch über keine Programmiererfahrung. Durch KI-gestützte Werkzeuge wie Replit oder Lovable ist es ihnen heute möglich, innerhalb kurzer Zeit funktionsfähige Anwendungen zu erzeugen, ohne eine einzige Zeile Code selbst zu schreiben oder zu verstehen. Diese Anwendungen nutzen die Erstellenden im Vertrauen darauf, sie seien sicher und produktionsreif. Tatsächlich betrachten sie den erzeugten Code jedoch nie; sie beurteilen die Anwendung ausschließlich anhand ihrer sichtbaren Funktionen. Der Quellcode bleibt für sie eine Blackbox, deren Qualität und Sicherheit sie nicht einschätzen können.

Die Relevanz dieser Entwicklung reicht über die Erstellung kleiner Hobby-Werkzeuge hinaus: KI-gestützte Entwicklungsplattformen richten sich auch an Unternehmen und werben mit Anwendungen für den produktiven Einsatz, wie etwa die Unternehmensbeispiele auf der Website von Lovable zeigen. [1] Damit gewinnen Anforderungen an Sicherheit, Zuverlässigkeit und Wartbarkeit an Bedeutung. Werden solche Anwendungen zudem als lokale Desktop-Anwendungen mit direktem Zugriff auf Dateisystem und Systemressourcen betrieben, müssen zusätzliche Sicherheitsanforderungen berücksichtigt werden. Eine Anwendung, die oberflächlich funktioniert, aber unbemerkt unsicher oder architektonisch mangelhaft ist, kann in einer solchen lokalen Laufzeitumgebung Schäden verursachen, die über die Zugriffsmöglichkeiten einer im Browser isolierten Anwendung hinausgehen. Genau an dieser Stelle setzt die vorliegende Arbeit an.

1.2 Problemstellung

Von technischen Laien instruierte generative KI-Modelle fokussieren sich nahezu ausschließlich auf die Umsetzung der gewünschten Funktionen. Fundamentale Qualitätsmerkmale des Software Engineerings nach ISO/IEC 25010 — Wartbarkeit, Trennung der Belange, Interoperabilität und Sicherheit — bleiben dabei regelmäßig unberücksichtigt. Auch die Produktdarstellung von Lovable betont die schnelle Umsetzung beschriebener Ideen und deren unmittelbare Bereitstellung. Zwar bewirbt die Plattform zugleich Sicherheitsfunktionen; aus der sichtbaren Funktionalität und den Produktversprechen allein lässt sich jedoch nicht ableiten, ob der erzeugte Code die Qualitätsanforderungen eines konkreten Einsatzszenarios erfüllt. [1] Der entstehende Code ist häufig unstrukturiert und stark gekoppelt, was spätere Wartung und Erweiterung erschwert. Da die Erstellenden nur die Funktionen, nicht aber den Code beurteilen, bleibt diese Diskrepanz für sie unsichtbar: Die Anwendung wirkt einsatzbereit, obwohl sie es nicht ist.

Besonders kritisch wird diese Diskrepanz, sobald eine solche Webanwendung in eine lokale Desktop-Umgebung überführt werden soll. Die Zielumgebung stellt völlig andere Anforderungen als die isolierte Sandbox des KI-Werkzeugs, und die vom Werkzeug getroffenen Architekturentscheidungen lassen sich

nicht unverändert übernehmen. Technisch wäre es mit vergleichsweise geringem Aufwand möglich, eine Webanwendung nur oberflächlich zu verpacken, etwa indem im Main-Prozess einer Electron-Anwendung eine lokale Web-API betrieben und das Frontend unverändert im Renderer ausgeliefert wird. Ein solcher Ansatz ermöglicht zwar die Ausführung auf dem Desktop, berücksichtigt für sich genommen jedoch nicht die zusätzlichen Sicherheitsanforderungen, die durch den Zugriff auf lokale Dateien und Systemressourcen entstehen. Ohne eine gezielte Anpassung der Vertrauensgrenzen und Zugriffsmöglichkeiten können Schwachstellen der ursprünglichen Webanwendung dadurch Auswirkungen auf das lokale System erhalten. Ziel der Arbeit ist deshalb nicht die bloße Ausführbarkeit auf dem Desktop, sondern eine sichere, architektonisch saubere Transformation. Für genau diese Klasse von Anwendungen fehlt bislang ein systematisches Vorgehen, um eine von Laien in Auftrag gegebene, KI-generierte Webanwendung sicher in eine lokale Desktop-Anwendung zu transformieren.

1.3 Fallstudie und Zielsetzung

Um diese Lücke zu schließen, entwickelt die vorliegende Arbeit einen methodischen Migrationspfad für die Überführung KI-generierter Webanwendungen in sichere Local-First-Desktop-Anwendungen. Validierungsobjekt ist eine konkrete Fallstudie: der Studienplaner, eine deutschsprachige Studienverwaltungsanwendung, die als Replit-Webprototyp vorliegt und in eine Electron-Anwendung überführt wird. Entscheidend ist dabei, wie dieser Prototyp entstanden ist: Er wurde bewusst so in Auftrag gegeben, wie es eine Person ohne technischen Hintergrund tun würde. Die beiden ursprünglichen Prompts beschreiben in überwiegend alltagssprachlicher Form gewünschte Funktionen und Nutzungsszenarien, etwa einen persönlichen Stundenplan und die Zusammenarbeit in Studiengruppen (Abschnitt A.1 und Abschnitt A.2). Sie enthalten auch Anforderungen an Anmeldung und Zugriffsschutz, machen jedoch keine Vorgaben zur Wahl eines UI-Frameworks oder einer konkreten Backend- oder Datenbanktechnologie. Der Studienplaner ist damit keine von einem erfahrenen Entwickler gestaltete Anwendung, sondern eine authentisch laienhaft erzeugte, und repräsentiert so genau die Klasse von Artefakten, auf die der Migrationspfad abzielt.

Die Wahl des Local-First-Paradigmas als Zielarchitektur ist dabei nicht beliebig. Local-First bündelt mehrere für die Migration KI-generierter Software zentrale Eigenschaften: Offline-Fähigkeit, Datensouveränität und eine vom Netzwerk entkoppelte, dauerhaft verfügbare Nutzung. Die Arbeit untersucht jedoch nicht sämtliche Aspekte dieses Paradigmas, sondern setzt einen besonderen Schwerpunkt auf die Synchronisierung lokal gespeicherter Daten. Damit soll veranschaulicht werden, dass die Migration in eine Desktop-Umgebung nicht auf den Umbau zu einer ausschließlich offline nutzbaren Anwendung beschränkt ist. Online-Funktionen können erhalten bleiben und mit lokaler Datenhaltung sowie einer auch ohne Netzwerk verfügbaren Kernfunktionalität verbunden werden.

1.4 Forschungsfrage

FF1: Wie können KI-generierte Webarchitekturen mithilfe eines framework-agnostischen Ansatzes systematisch in sichere Local-First-Desktop-Anwendungen überführt werden?

1.5 Aufbau der Arbeit

Der weitere Aufbau der Arbeit orientiert sich am Design-Science-Research-Prozess und gliedert sich in sieben weitere Kapitel. Kapitel 2 legt den theoretischen Hintergrund, indem es KI-gestützte Entwicklungswerkzeuge, die Besonderheiten von Web-zu-Desktop-Architekturen, das Local-First-Paradigma sowie sicherheitsrelevante Grundlagen einordnet und die bestehende Forschungslücke herausarbeitet. Kapitel 3 stellt den methodischen Rahmen der Arbeit vor, begründet die Wahl des Design-Science-Research-Ansatzes und skizziert den eigenen Forschungsprozess. Kapitel 4 widmet sich der Problemanalyse: Zunächst wird die Softwarelandschaft KI-gestützter Entwicklungswerkzeuge, etwa Ansätze wie Replit oder Lovable, betrachtet, bevor in die Tiefe des gewählten Ansatzes, der Replit-Umgebung und des daraus resultierenden Artefakts, eingestiegen wird. Kapitel 5 behandelt Entwurf

und Entwicklung des Artefakts, also die konzeptionelle Architektur und die praktische Umsetzung der Electron-Anwendung. Kapitel 6 stellt die Evaluation und Demonstration des entwickelten Migrationspfads anhand der Fallstudie in den Mittelpunkt. Kapitel 7 diskutiert die gewonnenen Erkenntnisse, reflektiert die verfolgten Lösungswege und ordnet die Ergebnisse im Vergleich zu bestehenden Ansätzen ein. Kapitel 8 fasst die zentralen Erkenntnisse zusammen und gibt einen Ausblick auf offene Fragestellungen und zukünftigen Forschungsbedarf.

Kapitel 2

Theoretischer Hintergrund

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Data are based in the repository[1]. The data is stored in the `data` directory, and the code for processing the data is stored in the `code` directory.

Tabelle 2.1 shows an example dataset that we will use for our experiments.

Name	**Value**	**Unit**
Temperature	23.5	°C
Pressure	1013	hPa
Humidity	45	%

Tabelle 2.1: Example Dataset

[1]https://github.com/th-nuernberg/thesis-template-typst

Kapitel 3

Methodisches Vorgehen

3.1 Forschungsansatz und Erkenntnisziel

Die Arbeit verfolgt einen gestaltungsorientierten Forschungsansatz. Im Mittelpunkt stehen die Entwicklung eines Migrationsleitfadens und dessen Anwendung auf den KI-generierten Webprototyp Studienplaner. Als Forschungsrahmen dient Design Science Research. Das Prozessmodell nach Peffers et al. unterscheidet Problemidentifikation und Motivation, Definition der Lösungsziele, Entwurf und Entwicklung, Demonstration, Evaluation sowie Kommunikation. [2]

Die Zuordnung zu diesem Rahmen strukturiert die wissenschaftliche Darstellung des tatsächlich durchgeführten Vorgehens. Sie bedeutet nicht, dass sämtliche technischen Schritte bereits zu Beginn festgelegt waren. Die Migration wurde schrittweise entwickelt; Erkenntnisse aus der Analyse und Umsetzung führten zu Anpassungen der Architektur und des weiteren Vorgehens. Die nachfolgend beschriebenen Phasen fassen diesen Prozess rückblickend zusammen.

Das Erkenntnisziel besteht darin, die für die Migration notwendigen Veränderungen nachvollziehbar zu beschreiben und daraus einen fallbezogenen Leitfaden abzuleiten. Die Electron-Anwendung dient als technischer Demonstrator. Der Leitfaden dokumentiert die Voraussetzungen, Entscheidungen, Transformationsschritte und Kontrollpunkte der Migration. Aussagen über seine Übertragbarkeit werden auf den untersuchten Fall begrenzt.

3.2 Untersuchungsgegenstand und Analyseverfahren

Ausgangspunkt war der auf Replit erzeugte Studienplaner. Wie in Kapitel 1 erläutert, erfolgte dessen Erstellung anhand funktionaler Beschreibungen ohne verbindliche Vorgaben zur technischen Architektur. Untersucht wurde damit ein KI-generiertes Ausgangsartefakt, dessen Komponenten für eine lokale Desktop-Laufzeit neu eingeordnet werden mussten.

Die beiden ursprünglichen Erstellungs-Prompts sind in Abschnitt A.1 und Abschnitt A.2 vollständig dokumentiert. Sie beschreiben in überwiegend alltagssprachlicher Form zunächst die persönliche Studienorganisation und anschließend die Zusammenarbeit in Studiengruppen. Neben funktionalen Wünschen nennen sie Anforderungen an Anmeldung und Zugriffsschutz, legen jedoch weder ein UI-Framework noch eine konkrete Backend- oder Datenbanktechnologie fest.

Die Untersuchung erfolgte anhand des Quellcodes, der Projektkonfiguration und der vorhandenen technischen Dokumentation. Dabei wurden Oberfläche, Schnittstellen, Geschäftslogik, Datenhaltung und externe Abhängigkeiten erfasst. Für die Migration war insbesondere zu klären, welche Funktionen unmittelbar von einem Backend oder einer Netzwerkverbindung abhingen, welche Bestandteile lokal weiterverwendet werden konnten und an welchen Stellen eine Anpassung der Verantwortlichkeiten erforderlich war.

Ergänzend wurde die offizielle Plattformdokumentation herangezogen, um die Wahl des Untersuchungsgegenstands einzuordnen. Replit und Lovable wurden hierzu beispielhaft hinsichtlich ihrer dokumentierten Backend-Ansätze betrachtet. Diese Betrachtung dient der Begründung des Untersuchungsumfangs; sie ist keine empirische Vergleichsstudie der Plattformen. Die Ergebnisse und der daraus abgeleitete Anforderungskatalog werden in Kapitel 4 dargestellt.

3.3 Ablauf der Entwicklung

Die technische Entwicklung gliederte sich in drei aufeinander aufbauende Phasen. Diese sind Arbeitsschritte innerhalb der Entwurfs- und Entwicklungsaktivität des Forschungsrahmens und bilden kein eigenständiges Forschungsmodell.

3.3.1 Phase 1: Überführung in eine lokal nutzbare Offline-Anwendung

Zunächst wurden die eingeschlossenen Kernfunktionen von der ursprünglichen Server-Laufzeit entkoppelt. Hierzu wurden die Komponenten auf eine Desktop-Architektur mit getrennten Verantwortlichkeiten für Darstellung, Geschäftslogik und Datenzugriff abgebildet. Die Datenhaltung wurde lokal bereitgestellt. Aufrufe der bisherigen Webschnittstellen wurden in einen begrenzten Kommunikationsweg zwischen Renderer, Preload und Main-Prozess überführt.

Ziel dieser Phase war die Entwicklung einer Electron-Anwendung, deren lokale Kernfunktionen ohne Netzwerkverbindung ausführbar sind und deren Architektur zugleich die Grundlage für die nachfolgenden Erweiterungen bildet. Die Verantwortlichkeiten und Schnittstellen wurden so gestaltet, dass Synchronisierung und weitere externe Dienste später integriert werden können, ohne die lokale Kernarchitektur erneut grundlegend umzustrukturieren. Damit verbindet diese Phase die Herstellung lokaler Nutzbarkeit mit der Vorbereitung einer schrittweise erweiterbaren Anwendung.

3.3.2 Phase 2: Ergänzung um Synchronisierung und weitere Online-Anbindungen

Aufbauend auf der lokal ausführbaren Kernanwendung wurden in der zweiten Phase die netzwerkabhängigen Funktionen betrachtet. Diese waren bei der Herstellung der Offline-Kernanwendung zunächst ausgeklammert worden. Ziel war es, externe Datenquellen und Dienste schrittweise in die vorbereitete Desktop-Architektur einzubinden und dadurch den angestrebten Funktionsumfang zu vervollständigen.

Die Wiederanbindung wurde anhand zweier unterschiedlich komplexer Fälle untersucht. Der Abruf eines Studiengangkatalogs dient als Beispiel für das Herunterladen externer Daten und deren Übernahme in die lokale Anwendung. Dieser Vorgang umfasst keine Synchronisierung und veranschaulicht zunächst einen einfachen Netzwerkzugriff. Darauf aufbauend wurde mit der gemeinsamen Aufgabenverwaltung in Studiengruppen ein Kollaborationsfall betrachtet. Hier werden lokal verarbeitete Änderungen über eine zusätzliche Synchronisierungsinfrastruktur zwischen den Beteiligten ausgetauscht. Dieser zweite Fall greift gezielt Eigenschaften des Local-First-Paradigmas auf.

3.3.3 Phase 3: Konsolidierte Prüfung und Härtung

Nach der funktionalen Umsetzung wurden die entstandenen lokalen und netzwerkbezogenen Schnittstellen systematisch hinsichtlich der Electron-Sicherheitsrichtlinien geprüft und erforderliche Anpassungen vorgenommen. Der Sicherheitsbegriff wird für diese Arbeit über die Umsetzung der anwendbaren Electron Security Guidelines einschließlich Eingabevalidierung und Maßnahmen gegen die Ausführung fremden Codes operationalisiert. [3]

Diese abschließende Phase bündelt die Prüfung der fertigen Anwendung. Sicherheitsrelevante Architekturentscheidungen, insbesondere die Trennung privilegierter Funktionen vom Renderer und die Begrenzung der Preload-Schnittstellen, betreffen bereits die vorausgehenden Phasen. Die Darstellung

einer abschließenden Härtung bedeutet daher nicht, dass Sicherheit erst nach Abschluss der Entwicklung berücksichtigt wird.

3.4 Ableitung des Leitfadens und Bewertung

Aus dem dokumentierten Migrationsprozess werden die Schritte des Leitfadens abgeleitet. Für jeden Schritt werden Ausgangszustand, Ziel, notwendige Änderungen und ein Kontrollpunkt beschrieben. Dabei werden architektonische Prinzipien von Anpassungen unterschieden, die an den konkreten Technologieeinsatz der Fallstudie gebunden sind. Die technischen Schritte und ihre Zuordnung zum Demonstrator werden in Kapitel 5 erläutert.

Die Bewertung orientiert sich am Anforderungskatalog aus Kapitel 4. Architektur- und Quellcodeinspektionen, vorhandene Service-Unit-Tests sowie begrenzte manuelle Demonstrationen bilden die vorgesehenen Nachweisformen. Für den Sicherheitsnachweis werden Anwendbarkeit, Umsetzung und Beleg jeder Electron-Empfehlung getrennt erfasst. Bereits vorliegende Sicherheitsberichte werden mit ihrem jeweiligen Prüfstand und ihren Nachweisgrenzen berücksichtigt.

Die Festlegung der Nachweise ist von den Ergebnissen ihrer Durchführung zu unterscheiden. Aus der Beschreibung des Verfahrens folgt keine pauschale Erfüllung der Anforderungen. Die Ergebnisdarstellung erfolgt in Kapitel 6. Ein weiterer Migrationsdurchlauf zur unabhängigen Erprobung des Leitfadens ist nicht Bestandteil der Arbeit. Die Bewertung stützt sich auf den dokumentierten Prozess und den vorhandenen Demonstrator; eine empirisch bestätigte Übertragbarkeit auf weitere Plattformen wird daraus nicht abgeleitet.

Kapitel 4

Analyse und Anforderungsdefinition

4.1 Vom übergeordneten Ziel zur Fallauswahl

Die in Kapitel 1 formulierte Forschungsfrage nimmt einen frameworkagnostischen Migrationsansatz als Ausgangsziel auf. Für dessen Bearbeitung ist zunächst zu bestimmen, welche Architekturmerkmale des Ausgangsartefakts die Transformation beeinflussen. Entscheidend ist nicht allein das verwendete Frontend-Framework, sondern auch, wo Geschäftslogik, Datenhaltung und Authentifizierung ausgeführt werden und welche dieser Bestandteile im verfügbaren Quellcode enthalten sind.

Zur Einordnung werden Replit und Lovable anhand ihrer offiziellen Dokumentation beispielhaft betrachtet. Replit beschreibt Webanwendungen als standardmäßig erzeugte Full-Stack-Anwendungen mit Frontend und Backend. API-Routen, Datenbank und serverseitige Logik werden nach dem Bedarf der Anwendung eingerichtet. Diese Dokumentation beschreibt das Plattformangebot; den konkreten Aufbau des Studienplaners belegt erst die Analyse des vorliegenden Artefakts. [4]

Lovable dokumentiert dagegen einen standardmäßig aktivierten verwalteten Cloud-Backend-Dienst auf Grundlage der Open-Source-Technologie von Supabase. Alternativ kann ein eigenes Supabase-Projekt angebunden werden. Datenbank, Authentifizierung, Dateispeicherung und serverseitige Funktionen werden damit als Backend-Dienste bereitgestellt. Diese dokumentierte Dienststruktur wird hier als Backend-as-a-Service eingeordnet. Daraus folgt jedoch nicht, dass jedes Lovable-Projekt dieselbe Architektur aufweist. [5]

Für die Migration ergibt sich daraus eine unterschiedliche Ausgangslage. Liegt die serverseitige Anwendungslogik als eigener Quellcode vor, kann ihre Übernahme und Anpassung an die lokale Laufzeit untersucht werden. Bei einer dienstgebundenen Architektur müssen dagegen auch die vom Dienst übernommenen Verantwortlichkeiten identifiziert und gegebenenfalls lokal nachgebildet werden. Eine reine Ersetzung des Frontend-Transports deckt diese zweite Ausgangslage nicht vollständig ab.

Die Entwicklung und Prüfung eines allgemeinen Leitfadens für beide Fälle würde den Umfang dieser Arbeit überschreiten. Die praktische Untersuchung wird daher auf den Replit-Prototyp Studienplaner begrenzt. Lovable dient zur dokumentationsgestützten Begründung dieser Eingrenzung; eine Lovable-Anwendung wird nicht migriert. Der resultierende Leitfaden kann potenziell übertragbare Prinzipien aufzeigen, wird jedoch nicht als validierte Lösung für beliebige Frameworks oder Plattformen ausgegeben.

4.2 Funktionen und Architektur des Ausgangsartefakts

4.2.1 Funktionaler Untersuchungsumfang

Der funktionale Ausgangspunkt ergibt sich aus den ursprünglichen Erstellungs-Prompts zur persönlichen Studienorganisation und zur Zusammenarbeit in Studiengruppen. Beide Prompts sind in Anhang A dokumentiert. Für die Migration werden sämtliche persönlichen Funktionsbereiche sowie Authentifizierung und ausgewählte Gruppenfunktionen betrachtet. Tabelle 4.1 konkretisiert diesen Untersuchungsumfang. Die Aufstellung bildet den funktionalen Bezugsrahmen; maßgeblich für den Demonstrator ist der in Abschnitt 4.3 definierte repräsentative Kernfunktionsumfang.

Bereich	**Betrachtete Funktionen**
Dashboard	Notendurchschnitt, gesammelte ECTS-Punkte, anstehende Fristen, Schnellzugriffe sowie letzte Aktivitäten und nächste Termine.
Kalender	Monatsansicht mit Tages-Popover, Termindetails und Löschen, farbliche Kennzeichnung der Terminarten, iCal-/ICS-Import, mehrtägige Termine und nach Priorität filterbare Listenansicht.
Noten	Semesterweise gruppierte Kurs-, Noten- und ECTS-Tabelle; Anlegen, Lesen, Bearbeiten und Löschen; CSV-Import mit Spaltenerkennung; ECTS-gewichteter Notendurchschnitt und Prüfung der Kurszugehörigkeit zum Studiengang.
Stundenplan	Raster mit fünf Wochentagen und sechs festen Hochschulzeitblöcken, Auswahl vorbereiteter Hochschulpläne, Hinzufügen und Entfernen von Kursen, visuelle Unterscheidung vorgeschlagener und eigener Einträge sowie Leeren des Plans mit Bestätigung.
Authentifizierung	Registrierung, Anmeldung und Abmeldung sowie die Zuordnung persönlicher Daten zur jeweiligen Identität.
Studiengruppen	Anlegen benannter Gruppen und Einladen weiterer Mitglieder; gemeinsame Aufgaben mit Fälligkeit, zuständiger Person und Status „offen“ beziehungsweise „erledigt“.

Tabelle 4.1: Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts.

Im Webprototyp wird die angemeldete Sitzung über Session-Cookies vermittelt. Bei der Migration sind die lokale Zuordnung persönlicher Daten und die für die Zusammenarbeit benötigte entfernte Authentifizierung gesondert zu betrachten. Die konkrete Umsetzung der Identitäts- und Sitzungsverwaltung wird in Kapitel 5 beschrieben.

Der Kollaborationsumfang wird auf Gruppenerstellung, Einladungen und gemeinsame Aufgaben begrenzt. Kommentare, gemeinsame Kalendertermine sowie zusätzliche Gruppeninhalte wie Links und Notizen werden nicht untersucht. Die Sichtbarkeit gemeinsamer Aufgaben bleibt auf die jeweiligen Gruppenmitglieder begrenzt; persönliche Noten und Termine werden durch die Gruppenzugehörigkeit nicht automatisch geteilt.

4.2.2 Technische Komponenten und Repository-Struktur

Der vorliegende Replit-Prototyp enthält eine React-Oberfläche, einen eigenständigen Express-API-Server, einen durch OpenAPI beschriebenen API-Vertrag sowie eine über Drizzle angebundene

PostgreSQL-Datenhaltung. Oberfläche, Backend und gemeinsam genutzte Bibliotheken sind im Projekt getrennt abgelegt. Die Datenbankanbindung erwartet eine konfigurierte entfernte Datenbankverbindung. Damit liegt im untersuchten Fall serverseitiger Quellcode vor, der für die Migration analysiert und angepasst werden kann. [6]

Die Schnittstellen zwischen Frontend und Backend werden in der Datei `lib/api-spec/openapi.yaml` als gemeinsamer API-Vertrag beschrieben. Dieser legt die Endpunkte sowie die Strukturen der Anfragen und Antworten fest. Auf dieser Grundlage erzeugt Orval API-Aufrufhilfen, React-Query-Hooks und Typdefinitionen für das Frontend sowie Zod-Schemata für die Validierung im Backend. Die generierten Bestandteile werden über gemeinsam genutzte Projektbibliotheken eingebunden. Der OpenAPI-Vertrag bildet damit die gemeinsame Grundlage für die Schnittstellendefinition auf beiden Seiten und wird bei der späteren Anpassung des Kommunikationswegs erneut aufgegriffen. [6]

Die Oberfläche in `artifacts/studienplaner/` verwendet React und Vite sowie Tailwind CSS und shadcn/ui für die Darstellung. Der Express-Server liegt in `artifacts/api-server/` und wird mit esbuild als ESM-Anwendung gebündelt. Die gemeinsam genutzten Bibliotheken sind unter `lib/` zusammengefasst: `db/` enthält das Drizzle-Schema und den PostgreSQL-Client, `api-spec/` den OpenAPI-Vertrag, `api-zod/` die daraus erzeugten Validierungsschemata und `api-client-react/` die mit Orval generierten React-Query-Aufrufhilfen. `artifacts/mockup-sandbox/` dient der Gestaltung und Erprobung von Oberflächen und wird nicht als fachlicher Bestandteil der Migration betrachtet. [6]

Abbildung 4.1 stellt die Komponenten und ihre Beziehungen dar. Die durchgezogenen Pfeile zeigen den Kommunikationsweg zur Laufzeit: Die React-Oberfläche ruft den Express-API-Server über HTTP auf; dessen Route-Handler greifen über Drizzle und den PostgreSQL-Client auf die Datenbank zu. Die gestrichelten Pfeile zeigen die Codegenerierung und Einbindung der aus dem OpenAPI-Vertrag erzeugten Bestandteile. Der Vertrag selbst ist damit eine Grundlage der Entwicklung und kein zusätzlicher Vermittler im Laufzeitpfad.

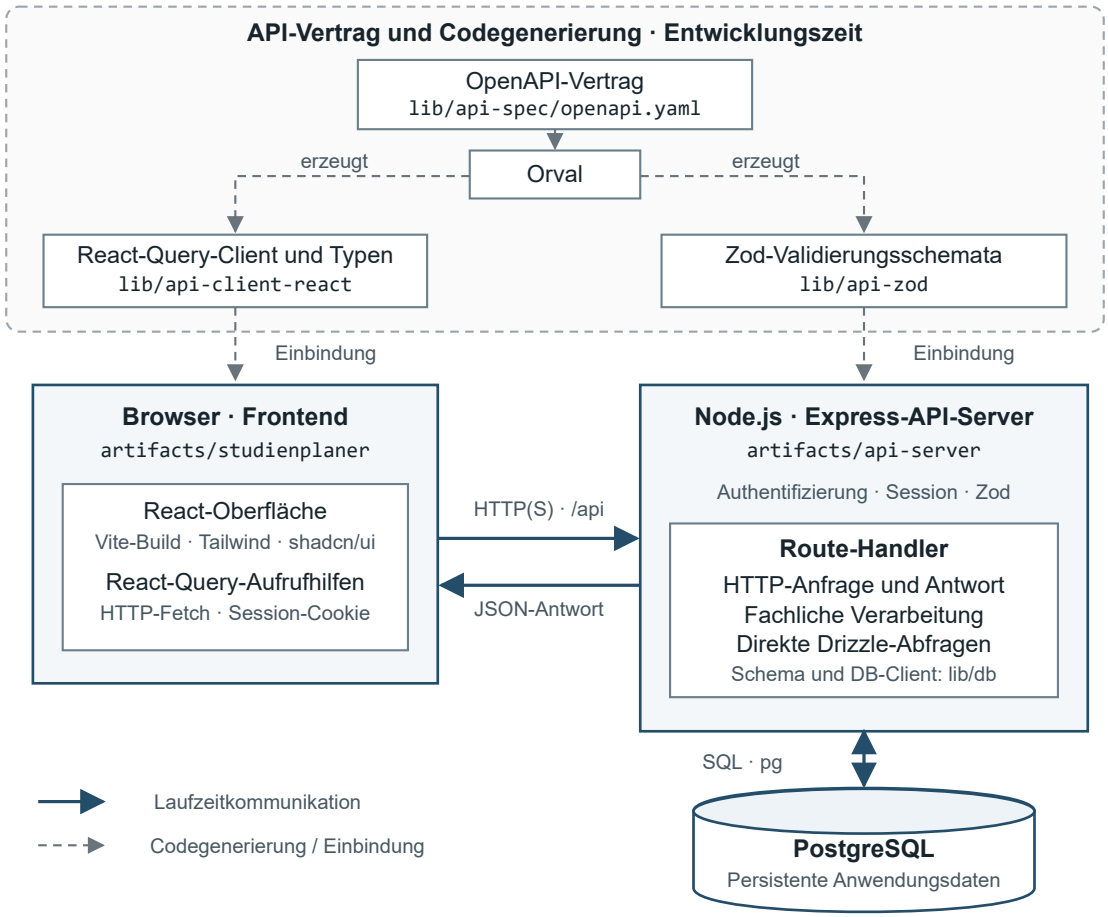


Abbildung 4.1: Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts.

Diese Projektaufteilung ist noch kein Nachweis einer durchgängig getrennten Dreischichtenarchitektur. Für die Transformation sind die tatsächlichen Verantwortlichkeiten der Module und ihre Abhängigkeiten maßgeblich. Das in Quellcode 4.1 gezeigte Beispiel veranschaulicht das im API-Server verwendete Verarbeitungsmuster innerhalb eines Route-Handlers.

```
router.get("/dashboard/upcoming", requireAuth, async (req, res): Promise<void> => {
  const userId = req.session.userId!;
  const now = new Date();
  const in7Days = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);

  const events = await db
    .select()
    .from(eventsTable)
    .where(
      and(
        eq(eventsTable.userId, userId),
        gte(eventsTable.startDate, now),
        lte(eventsTable.startDate, in7Days),
      ),
    )
    .orderBy(eventsTable.startDate);

  res.json(events);
});
```

Quellcode 4.1: Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt.

Der Handler liest nach der Authentifizierungsprüfung die Benutzerkennung aus der Sitzung, selektiert Termine für die kommenden sieben Tage und liefert das Abfrageergebnis als JSON zurück. Damit verbindet er HTTP-Verarbeitung, Fachlogik und Datenzugriff. Die Migration erfordert neben der Entkopplung vom HTTP-Transport eine architektonische Trennung in Controller (Schnittstellenverarbeitung), Service (Fachlogik) und Repository (Datenzugriff). Diese Aufteilung ermöglicht eine isolierte Prüfung der Geschäftslogik mit austauschbaren Datenzugriffen und erleichtert die Wartung sowie Anpassungen an Transport und Persistenz.

Das zentrale Migrationsproblem besteht somit in der Verlagerung der bisher servergebundenen Funktionen in die lokale Desktop-Laufzeit bei gleichzeitigem Erhalt der eingeschlossenen Kernfunktionen. Die Oberfläche soll möglichst weitgehend weiterverwendet werden. Technisch notwendige Anpassungen betreffen insbesondere den Aufrufweg zur Geschäftslogik, das Datenbankschema, die Sitzungsverwaltung und die Behandlung externer Dienste.

4.2.3 Lokale Funktionen und exemplarische Online-Anbindungen

Die persönlichen Funktionen ermöglichen die Untersuchung unterschiedlicher lokaler Interaktionsformen. Die CRUD-Funktionen der Noten-, Termin- und Stundenplanverwaltung stehen für die Verarbeitung von Benutzereingaben und den Zugriff auf persistente Anwendungsdaten. Die CSV- und iCal-/ICS-Importe ergänzen diesen Umfang um die Übernahme dateibasierter Daten. Dabei müssen externe Daten eingelesen, interpretiert, validiert und auf das interne Datenmodell abgebildet werden. Der Untersuchungsumfang umfasst somit sowohl die unmittelbare Bearbeitung von Daten über die Oberfläche als auch deren Übernahme aus Dateien. Diese Interaktionsformen bilden die Grundlage für die lokale Migration, bevor zusätzliche Netzwerkzugriffe und Synchronisierung betrachtet werden.

Für die Zielanwendung werden zwei Online-Anbindungen betrachtet: der Abruf des Studiengangkatalogs und die Zusammenarbeit im ausgewählten Gruppenbereich. Die übrigen eingeschlossenen Funktionen werden so eingeordnet, dass ihre Datenhaltung und Verarbeitung lokal erfolgen können. Dazu zählen auch die persönliche Datenzuordnung und die lokalen Dateiimporte. Nach einem Katalogabruf stehen die übernommenen Daten der lokalen Verarbeitung zur Verfügung; der Abruf selbst benötigt eine Verbindung.

Der Katalog dient als Beispiel für das Herunterladen, Prüfen und lokale Übernehmen externer Daten. Der Gruppenbereich ergänzt dieses Muster um entfernte Authentifizierung, Mitgliedschaft und Einladungen sowie die Synchronisierung lokal verarbeiteter Aufgabenänderungen. Innerhalb dieses Bereichs ist zwischen synchronisierbaren Aufgabenoperationen und unmittelbar netzwerkabhängigen Verwaltungsoperationen zu unterscheiden.

Diese Auswahl deckt zwei für den Leitfaden relevante Integrationsmuster ab: externe Datenübernahme und Local-First-Kollaboration. Weitere Funktionen mit vergleichbaren Anforderungen können auf denselben architektonischen Prinzipien aufbauen. Daraus ergibt sich die Annahme, dass solche Erweiterungen überwiegend eine erneute Anwendung der beschriebenen Muster erfordern. Eine Übertragbarkeit auf Funktionen mit zusätzlichen Anforderungen wird dadurch nicht nachgewiesen; diese wäre im jeweiligen Fall gesondert zu prüfen.

4.3 Zielarchitektur und Sicherheitsbegriff

Ziel der Migration ist eine architektonisch nachvollziehbare Desktop-Anwendung, deren Struktur zugleich die Grundlage des Migrationsleitfadens bildet. Architektonische Qualität wird dabei durch klar zugeordnete Verantwortlichkeiten, definierte Schnittstellen und kontrollierte Abhängigkeiten konkretisiert. Wiederkehrende Aufgaben sollen nach einheitlichen Mustern umgesetzt werden. Dadurch können die Transformationsschritte verständlich beschrieben und auf weitere Funktionen mit vergleichbaren Anforderungen bezogen werden.

Die Zielarchitektur soll Darstellung, Geschäftslogik und Datenzugriff trennen. Die Verarbeitung fachlicher Aufrufe soll in Controller, Services und Repositories gegliedert werden. Der Renderer soll die Oberfläche bereitstellen; privilegierte Operationen sollen über begrenzte Preload-Schnittstellen an den Main-Prozess vermittelt werden. Die Kommunikation soll über Interprozesskommunikation erfolgen, ohne hierfür einen lokalen HTTP-Server zu betreiben. Die Prozessaufteilung und die fachliche Schichtentrennung ergänzen sich, sind jedoch nicht gleichzusetzen.

Funktional soll ein zusammenhängender, sinnvoll nutzbarer Kernumfang der ursprünglichen Anwendung erhalten bleiben. Entscheidend ist die Abdeckung der unterschiedlichen Interaktionsformen: Oberflächeninteraktionen mit lokaler Datenverarbeitung, das Einlesen und Validieren von Dateien sowie Netzwerkanfragen und die Synchronisierung gemeinsamer Aufgaben. Dashboard, Kalender, Noten und Studiengruppen bilden den fachlichen Kontext dieser allgemeineren Anwendungsaufgaben. Die vollständige Übernahme jeder CRUD-Variante ist demgegenüber kein eigenständiges Ziel. Der Kernumfang muss für die betrachteten Interaktionsformen vollständige Verarbeitungspfade einschließlich Datenzuordnung, Validierung und Fehlerbehandlung bereitstellen.

Lokale Datenhaltung bildet die Grundlage der Offline-Kernfunktionen. Local-First verbindet lokale Nutzbarkeit mit weitergehenden Eigenschaften wie Kollaboration und Kontrolle über die eigenen Daten. [7] Die Arbeit greift daraus gezielt die lokale Verarbeitung und die ergänzende Synchronisierung im ausgewählten Gruppenfeature auf. Der Katalogabruf ergänzt diesen Umfang als Beispiel einer externen Datenübernahme.

Erweiterbarkeit bedeutet in diesem Zusammenhang, dass eine Person nach Einarbeitung in die dokumentierten Muster zusätzliche Funktionen mit vergleichbaren Anforderungen entsprechend umsetzen kann. Die vorhandenen Beispiele müssen deshalb sowohl den fachlichen Ablauf als auch die erforderlichen Validierungen, Berechtigungsprüfungen und Sicherheitsgrenzen nachvollziehbar zeigen. Das Fortführen dieser Muster soll zu einer konsistenten Anwendung führen, ohne für jede weitere Funktion einen neuen Architekturansatz entwickeln zu müssen.

Die Anwendung soll außerdem strukturell auf spätere Tests vorbereitet sein. Fachliche Services müssen unabhängig von Oberfläche und Transport prüfbar sein; Datenzugriffe und externe Dienste sollen

für diese Tests durch geeignete Testimplementierungen ersetzt werden können. Zusätzliche Testfälle und Funktionen innerhalb der betrachteten Interaktionsformen sollen sich ergänzen lassen, ohne zuvor die Schichtentrennung oder die grundlegenden Abhängigkeiten neu ordnen zu müssen. Damit wird die Vermeidung solcher nachgelagerten Strukturkorrekturen als Ziel formuliert; die tatsächliche Testabdeckung wird gesondert bewertet.

Unter einer sicheren beziehungsweise gehärteten Electron-Anwendung versteht diese Arbeit die Umsetzung der anwendbaren Electron Security Guidelines im festgelegten Prüfumfang. Dazu gehören insbesondere die Isolation des Renderers, begrenzte privilegierte Schnittstellen, die Prüfung von IPC-Absendern und Eingaben sowie Beschränkungen für nachgeladene Inhalte, Navigation und zusätzliche Fenster. [3] Der Begriff bezeichnet damit einen konkret prüfbaren Richtlinienrahmen und keinen Nachweis der Abwesenheit sämtlicher Schwachstellen.

4.4 Anforderungskatalog

Die Anforderungen verbinden die Fallauswahl, das Architekturziel und den festgelegten funktionalen sowie sicherheitsbezogenen Umfang. Die Kennungen A1–A9 werden für die Zuordnung der Migrationsschritte und die spätere Evaluation verwendet. Die nachfolgend genannten Bewertungsansätze legen fest, worauf ein Nachweis Bezug nehmen muss; konkrete Szenarien und Ergebnisse werden in Kapitel 6 dokumentiert.

4.4.1 A1: Lauffähige Desktop-Anwendung

Das Replit-Artefakt muss in eine lauffähige Electron-Anwendung überführt werden. Bewertet werden der Start einer gebauten Anwendung und die Ausführbarkeit der eingeschlossenen Kernfunktionen. Die untersuchte Version und die Laufzeitumgebung sind im Nachweis festzuhalten.

4.4.2 A2: Dreischichtenarchitektur

Darstellung, Geschäftslogik und Datenzugriff müssen getrennte Verantwortlichkeiten besitzen; fachliche Aufrufe sind entsprechend in Controller, Services und Repositories zu gliedern. Abhängigkeiten sind so zu gestalten, dass die Geschäftslogik unabhängig von Oberfläche und Transport mit austauschbaren Datenzugriffen geprüft werden kann. Der Nachweis erfolgt durch den Abgleich des Architekturmodells mit den tatsächlichen Modulen und einem beispielhaften fachlichen Ablauf. Die Electron-Prozesse sind dabei nicht mit den fachlichen Schichten gleichzusetzen; Geschäftslogik und Datenzugriff können beide im Main-Prozess liegen.

4.4.3 A3: Begrenzung des Refactoring-Aufwands

Bestandteile des Ausgangsartefakts sollen soweit möglich weiterverwendet werden. Notwendige Anpassungen und Ersetzungen sind nach Komponenten beziehungsweise Schichten zu dokumentieren und zu begründen. Diese Anforderung verlangt nachvollziehbare Eingriffe, jedoch keinen Nachweis eines global minimalen Entwicklungsaufwands.

4.4.4 A4: Lokale Kommunikation ohne HTTP-Wrapper

Die lokale Anwendungslogik darf nicht lediglich durch den Weiterbetrieb des ursprünglichen HTTP-Backends eingebettet werden. Der Nachweis beschreibt den tatsächlichen Aufrufweg über Preload und IPC und zeigt, dass für lokale fachliche Operationen kein HTTP-Server erforderlich ist. Externe HTTPS-Anbindungen bleiben davon unberührt.

4.4.5 A5: Lokale Datenhaltung und Offline-Kernfunktionen

Die für den Kernumfang ausgewählten lokalen Funktionen aus Abschnitt 4.2.1 und die lokale Identitätsverwaltung müssen ohne Netzwerkverbindung nutzbar sein. Änderungen müssen lokal gespeichert werden und einen Neustart der Anwendung überdauern. Für Funktionen mit Katalogdaten wird

dabei ein bereits lokal verfügbarer Datenbestand vorausgesetzt. Der Nachweis muss zwischen diesen Funktionen und ausdrücklich netzwerkabhängigen Operationen unterscheiden.

4.4.6 A6: Local-First-Kollaboration am Gruppenfeature „Aufgaben“

Aufgabenänderungen müssen im festgelegten Kollaborationsumfang lokal verarbeitet und bei verfügbarer Verbindung übermittelt werden können. Der Nachweis beschreibt lokale Speicherung und anschließende Synchronisierung als getrennte Vorgänge. Gruppenerstellung und Einladungsfunktionen gehören zum Untersuchungsumfang; ihre unmittelbar netzwerkabhängigen Operationen sind gesondert auszuweisen. Kommentare sind ausgeschlossen. Eine Mehrgeräte-Synchronisierung oder gemeinsame Echtzeitbearbeitung von Dokumenten wird nicht verlangt.

4.4.7 A7: Härtung gemäß Electron Security Guidelines

Die offizielle Sicherheitscheckliste muss vollständig auf Anwendbarkeit geprüft werden. Für anwendbare Empfehlungen sind Umsetzung, Beleg und Nachweisebene zu dokumentieren. Eingabevalidierung und Maßnahmen gegen fremden Code werden durch geeignete vorhandene Tests und ausgewählte negative Laufzeitfälle ergänzt. Eine fehlende Prüfung darf nicht als Erfüllung gewertet werden; begrenzt belegte Aussagen müssen ihre Nachweisgrenze nennen.

4.4.8 A8: Repräsentativer Kernfunktionsumfang und Erweiterbarkeit

Der Demonstrator muss einen zusammenhängenden Kernfunktionsumfang mit repräsentativen Abläufen für Oberflächeninteraktionen, lokale Persistenz, Dateiimporte und Netzwerkanbindungen bereitstellen. Gruppenerstellung, Einladungen und synchronisierte Aufgaben bleiben Bestandteil dieses Umfangs. Die vollständige Übernahme jeder CRUD-Variante wird nicht verlangt. Der Nachweis benennt die tatsächlich übernommenen Funktionen, vergleicht deren Verhalten mit dem Ausgangsartefakt und ordnet ihnen die verwendeten Architektur- und Sicherheitsmuster zu. Er muss nachvollziehbar machen, wie weitere Funktionen mit vergleichbaren Anforderungen auf diesen Mustern aufbauen können. Änderungen der Identitäts- und Sitzungsverwaltung sind zu begründen. Vorhandene Service-Unit-Tests ergänzen den Nachweis für die tatsächlich geprüfte Logik.

4.4.9 A9: Nachvollziehbarer Migrationsleitfaden

Der Leitfaden muss Voraussetzungen, Reihenfolge, Änderungen, Entscheidungen und Kontrollpunkte der durchgeführten Migration beschreiben. Seine Schritte müssen dem Demonstrator zugeordnet werden können. Projektspezifische Anpassungen und mögliche übertragbare Prinzipien sind kenntlich zu machen. Ein zusätzlicher unabhängiger Reproduktionsdurchlauf ist kein Bestandteil dieser Anforderung.

4.5 Abgrenzung des Untersuchungsumfangs

Untersucht werden die Migration und Härtung der Desktop-Anwendung anhand eines repräsentativen Kernumfangs. Dieser umfasst drei Interaktionsformen:

- **CRUD-Oberflächeninteraktionen:** Anlegen, Lesen, Bearbeiten und Löschen von Anwendungsdaten über die Oberfläche, einschließlich Eingabevalidierung, fachlicher Verarbeitung und lokaler Persistenz. Die Noten-, Termin- und Stundenplanverwaltung liefern hierfür die betrachteten Beispiele.
- **Dateiverarbeitung:** Einlesen, Parsen und Validieren lokaler CSV- und iCal-/ICS-Dateien sowie die Übernahme ihrer Inhalte in das interne Datenmodell. Betrachtet wird damit der vollständige Verarbeitungspfad eines Dateiimports.
- **Netzwerkinteraktionen:** Abruf und Verarbeitung externer Katalogdaten sowie die Anbindung der Gruppenfunktionen mit Authentifizierung, Gruppenerstellung, Einladungen und Synchronisie-

rung gemeinsamer Aufgaben. Unmittelbare Online-Anfragen und nachgelagerte Synchronisierung werden unterschieden.

Der Sicherheitsumfang bezieht sich auf die Desktop-Anwendung und ihre Schnittstellen. Eingabevalidierung, anwendungsinterne Zugriffskontrollen und die sichere Anbindung sowie Verarbeitung externer Daten bleiben Bestandteil der Untersuchung.

Nicht untersucht werden die interne Sicherheit und der sichere Betrieb der entfernten Dienste, die Synchronisierung oder die Bereitstellung von Nutzdaten übernehmen. Die Härtung ihrer Server, Datenbanken und sonstigen Infrastruktur liegt außerhalb des Umfangs. Die Nutzung solcher Dienste begründet somit keine Aussage über deren eigene Sicherheit.

Ebenfalls ausgeschlossen ist der Schutz der lokalen Datenbank innerhalb des Betriebssystems, insbesondere die Verschlüsselung der Datenbankdatei und ihre Absicherung gegen Zugriffe anderer lokaler Prozesse oder Betriebssystembenutzer. Die Datenzuordnung und Zugriffskontrolle innerhalb der Anwendung sowie die Validierung vor Datenbankoperationen bleiben davon unberührt.

Die Local-First-Demonstration bleibt auf gemeinsame Aufgaben und die zugehörige Gruppen- und Einladungsverwaltung begrenzt. Kommentare, weitere gemeinsame Gruppeninhalte, die Synchronisierung persönlicher Daten über mehrere Endgeräte, gemeinsame Echtzeitbearbeitung von Dokumenten und Bucket-Dateifunktionen sind ausgeschlossen.

Frontend-Unit-Tests, automatisierte End-to-End-Tests und ein umfassender Sicherheitsvergleich zwischen Web- und Desktop-Laufzeit sind ausgeschlossen. Diese Abgrenzungen beschränken den Geltungsbereich der Ergebnisse und gelten nicht als unerfüllte Anforderungen.

Kapitel 5

Entwurf, Migration und Migrationsleitfaden

Dieses Kapitel dokumentiert den durchgeführten Migrationsprozess und die dabei getroffenen Entscheidungen. Die Darstellung beginnt mit der Vorbereitung der lokalen Datenhaltung und folgt anschließend den weiteren Transformationsschritten. Auf dieser Grundlage werden die Handlungsempfehlungen des Migrationsleitfadens abgeleitet. Die Beschreibung der konkreten Umsetzung und die daraus gewonnenen Empfehlungen sind dabei zu unterscheiden.

5.1 Struktur des Migrationsleitfadens

Der Leitfaden ordnet die Migration in drei Phasen: die Überführung in eine lokal nutzbare Offline-Anwendung, die Ergänzung um Synchronisierung und weitere Online-Anbindungen sowie die konsolidierte Härtung. Vorausgesetzt wird die Analyse des Ausgangsartefakts aus Kapitel 4. Das dort erstellte Komponentenbild und der Anforderungskatalog legen fest, welche Funktionen übernommen und welche Verantwortlichkeiten verändert werden müssen.

Die Phasen unterscheiden Zielzustände der Entwicklung. Sie behaupten weder eine von Beginn an vollständig festgelegte Projektplanung noch eine vollständige Trennung aller Arbeitsschritte. Insbesondere beeinflussen die Sicherheitsanforderungen bereits den ersten Architekturentwurf. Jede Phase beschreibt deshalb neben ihrem funktionalen Ziel auch die relevanten Schnittstellen und einen Kontrollpunkt für den erreichten Stand.

Phase	Zielzustand	Kontrollpunkt
1	Lokale Kernfunktionen mit lokaler Datenhaltung und begrenzter IPC-Kommunikation.	Kernabläufe ohne Netzwerk ausführen; Änderungen nach Neustart erneut lesen.
2	Synchronisierte Aufgaben und gesonderte Online-Anbindungen.	Lokale Speicherung, spätere Übermittlung und Verhalten bei Netzwerkausfall unterscheiden.
3	Geprüfte Härtung im festgelegten Electron-Sicherheitsrahmen.	Anwendbarkeit, Umsetzung und Nachweis jeder Empfehlung dokumentieren.

Tabelle 5.1: Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens.

5.2 Phase 1: Von der Webanwendung zur Offline-Kernanwendung

5.2.1 Initialisierung des Electron-Projekts und Übernahme der Datenbankmodule

Die Entscheidung für Electron orientierte sich an Anforderung A3, vorhandene Komponenten soweit möglich weiterzuverwenden. Das Backend des Ausgangsartefakts war in TypeScript für Node.js implementiert. [6] Im Main-Prozess von Electron steht ebenfalls eine Node.js-Laufzeit zur Verfügung. [8] Damit konnten die vorhandene fachliche Logik und ihre JavaScript- beziehungsweise TypeScript-Bibliotheken als Ausgangsbasis für die lokale Umsetzung dienen.

Tauri wurde als alternative Desktop-Laufzeit erwogen. Sein privilegierter Core-Prozess basiert auf Rust, während die Oberfläche weiterhin Webtechnologien verwenden kann. [9] Eine Übernahme der Backend-Funktionen in diesen Rust-Core hätte eine Portierung der TypeScript-Implementierung erfordert. Dabei wären neben der Syntax auch Sprachkonzepte, Datentypen, Fehlerbehandlung und Bibliotheksanbindungen anzupassen gewesen. Insbesondere hätte der mit Drizzle implementierte Datenzugriff durch eine entsprechende Rust-Implementierung ersetzt werden müssen. Der erwartete zusätzliche Aufwand sprach aus Sicht des Projekts gegen diesen Weg und für die Beibehaltung der bestehenden Implementierungssprache mit Electron.

Ein vollständiger Sprachwechsel ist bei Tauri allerdings keine zwingende Voraussetzung: Die offizielle Dokumentation beschreibt auch die Einbindung einer Node.js-Anwendung als separaten Sidecar-Prozess. [10] Dieser Ansatz ermöglicht die Weiterverwendung vorhandener Backend-Logik, erfordert jedoch die Einbindung und Auslieferung eines zusätzlichen Prozesses sowie dessen Kommunikation mit der übrigen Anwendung. Electron wurde gewählt, um die übernommenen Backend-Komponenten unmittelbar im Main-Prozess auszuführen. Die Entscheidung beruht auf der erwarteten Begrenzung der Eingriffe in das Ausgangsartefakt; ein vergleichender Nachweis des Migrationsaufwands gegenüber Tauri wurde nicht erbracht.

Zu Beginn wurde ein neues Electron-Projekt mit dem auf electron-vite beruhenden Projektgenerator angelegt. Gewählt wurde die Vorlage für React und TypeScript, da das Frontend des Replit-Prototyps bereits React, TypeScript und Vite verwendete. [6] Der Projektgenerator stellt hierfür die Vorlage `react-ts` bereit. [11]

Als Alternative wurde Electron Forge anhand seiner Dokumentation betrachtet. Die dort beschriebene Einrichtung von React mit TypeScript geht von der Vorlage `webpack-typescript` aus. [12] Forge bietet außerdem die Vorlagen `vite` und `vite-typescript` an. [13] Für electron-vite sprach im vorliegenden Projekt die unmittelbar verfügbare Kombination aus Vite, React und TypeScript. Der Entscheidung lag die Annahme zugrunde, dass die Beibehaltung dieser Technologien weniger Anpassungen an der Oberfläche und ihrer Build-Konfiguration erfordert als ein gleichzeitiger Wechsel des Build-Werkzeugs. Diese Annahme wurde nicht durch eine vergleichende Migration mit Electron Forge und Webpack evaluiert. Die Wahl von electron-vite wird daher als projektbezogene Entscheidung dokumentiert; ein geringerer Migrationsaufwand gegenüber den Alternativen ist damit nicht nachgewiesen.

Anschließend wurden die Drizzle-Schemadefinitionen aus der gemeinsamen Datenbankbibliothek des Ausgangsartefakts in den Main-Prozess übernommen. Im Zielprojekt liegen sie unter `src/main/db/schema`. Dies betrifft die Definitionen für Benutzer, Stundenpläne, Studiengänge, Gruppen, Noten, Termine und das Aktivitätsprotokoll sowie deren gemeinsames Exportmodul. Auch die bestehende Drizzle-Kit-Konfiguration wurde als Ausgangspunkt übernommen; im Zielprojekt befindet sich `drizzle.config.ts` im Projektstamm. Der Datenbankzugriff wurde damit dem Main-Prozess zugeordnet. [6], [14]

Für die ersten lokalen Entwicklungsschritte wurde die bisherige Verbindung zu PostgreSQL durch den Zugriff auf eine SQLite-Datei im Projektverzeichnis ersetzt. In der Drizzle-Konfiguration wurde

der Dialekt von `postgresql` auf `sqlite` umgestellt. Die bisher für die entfernte Datenbank verwendete Umgebungsvariable `DATABASE_URL` entfiel für diesen lokalen Datenbankzugriff.

Mit dem Wechsel der Ausführungsumgebung verschob sich auch die Verantwortung für die Datenbankverwaltung: Die Desktop-Anwendung übernimmt im Main-Prozess das Anlegen, Öffnen und Migrieren der lokalen Benutzerdatenbank. Die Datenbankdatei wird zur Laufzeit im anwendungsspezifischen Benutzerdatenverzeichnis außerhalb des Projekt- und Installationsverzeichnisses angelegt. [14] Hierfür wird der von Electron bereitgestellte Pfad `userData` verwendet. [15] Dieser Speicherort trennt veränderliche Benutzerdaten von den ausgelieferten Programmdateien. Die Datenhaltung benötigt dadurch keine Schreibrechte im Installationsverzeichnis und bleibt vom Austausch der Programmdateien bei einem Update getrennt. Ausgeliefert werden die Migrationsdateien, mit denen die Anwendung eine neue Datenbank einrichtet oder eine vorhandene aktualisiert. [14]

5.2.2 Anpassung der Tabellendefinitionen an SQLite

Die Übernahme der Dateien genügte noch nicht, da die bestehenden Definitionen PostgreSQL-spezifische Datentypen und Tabellenkonstruktoren verwendeten. Diese wurden durch die entsprechenden SQLite-Konstrukte ersetzt. Die fachlichen Tabellen- und Spaltennamen sowie die vorhandenen Pflichtfelder und Standardwerte wurden dabei möglichst beibehalten. Dieser Ansatz begrenzte die Änderungen am Datenmodell und entsprach dem in Kapitel 4 festgelegten Ziel, vorhandene Komponenten mit möglichst geringem Refactoring-Aufwand weiterzuverwenden.

Die Anpassung lässt sich am Stundenplanschema nachvollziehen. Quellcode 5.1 zeigt die ursprüngliche Definition der Tabelle `timetable_entries`; Quellcode 5.2 zeigt ihre Überführung in das SQLite-Schema des Demonstrators. Beide Ausschnitte lassen die übrigen Tabellen und die abgeleiteten Validierungsschemata weg; die Zeilenumbrüche wurden für die Darstellung angepasst. [6], [14]

```
import {
  pgTable, serial, text, integer, boolean, pgEnum
} from "drizzle-orm/pg-core";

export const weekDayEnum = pgEnum("week_day", [
  "monday", "tuesday", "wednesday", "thursday", "friday"
]);
export const timeSlotEnum = pgEnum("time_slot", [
  "slot1", "slot2", "slot3", "slot4", "slot5", "slot6"
]);

export const timetableEntriesTable = pgTable("timetable_entries", {
  id: serial("id").primaryKey(),
  courseName: text("course_name").notNull(),
  weekDay: weekDayEnum("week_day").notNull(),
  timeSlot: timeSlotEnum("time_slot").notNull(),
  room: text("room"),
  professor: text("professor"),
  isCustom: boolean("is_custom").notNull().default(true),
  color: text("color"),
  userId: integer("user_id").notNull(),
});
```

Quellcode 5.1: Stundenplaneinträge im PostgreSQL-Schema des Ausgangsartefakts (Ausschnitt).

```ts
import { sqliteTable, integer, text } from 'drizzle-orm/sqlite-core'

export const weekDays = [
  'monday', 'tuesday', 'wednesday', 'thursday', 'friday'
] as const
export type WeekDay = (typeof weekDays)[number]

export const timeSlots = [
  'slot1', 'slot2', 'slot3', 'slot4', 'slot5', 'slot6'
] as const
export type TimeSlot = (typeof timeSlots)[number]

export const timetableEntriesTable = sqliteTable('timetable_entries', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  courseName: text('course_name').notNull(),
  weekDay: text('week_day', { enum: weekDays }).notNull(),
  timeSlot: text('time_slot', { enum: timeSlots }).notNull(),
  room: text('room'),
  professor: text('professor'),
  isCustom: integer('is_custom', { mode: 'boolean' })
    .notNull().default(true),
  color: text('color'),
  userId: integer('user_id').notNull()
})
```

Quellcode 5.2: Stundenplaneinträge im SQLite-Schema des Demonstrators (Ausschnitt).

An die Stelle von `pgTable` trat `sqliteTable`. Der automatisch erzeugte Primärschlüssel wurde von `serial` auf einen Integer-Primärschlüssel mit `autoIncrement` umgestellt. Das boolesche Feld `isCustom` wird in SQLite als Integer gespeichert; Drizzles Modus `boolean` erhält dabei die Verwendung boolescher Werte im TypeScript-Code. [16]

Die PostgreSQL-Enums für Wochentage und Zeitblöcke wurden durch konstante Wertelisten und daraus abgeleitete TypeScript-Union-Typen ersetzt. Die Spalten verwenden diese Listen über die `enum`-Option des Text-Konstruktors. Diese Option unterstützt die Typableitung, erzwingt jedoch zur Laufzeit keinen zulässigen Wertebereich in der Datenbank. [16] Die erhaltene Typisierung ist daher nicht mit der bisherigen Datenbankprüfung gleichzusetzen; eine Prüfung eingehender Werte bleibt erforderlich. Die aus den Tabellen abgeleiteten Insert- und Select-Typen sowie die Erzeugung der Zod-Schemata konnten weiterhin nach dem vorhandenen Muster verwendet werden.

5.2.3 Anpassung der Datenbankabhängigkeiten

Für die benötigten Pakete wurde zunächst die Abhängigkeitskonfiguration des Ausgangsartefakts herangezogen. Die angegebenen Versionsbereiche für `drizzle-orm` (`^0.45.2`), `drizzle-kit` (`^0.31.10`) und `drizzle-zod` (`^0.8.3`) wurden übernommen. Der PostgreSQL-Treiber `pg` wurde für die lokale Datenhaltung durch `@libsql/client` ersetzt; im Demonstrator ist hierfür der Versionsbereich `^0.18.0` angegeben. [6], [14] Damit blieb Drizzle als Abstraktion für Schema und Datenzugriff erhalten, während der verwendete Treiber wechselte.

Die Verantwortlichkeiten der Pakete sind dabei zu unterscheiden: Der libSQL-Client öffnet die lokale Datenbankdatei, Drizzle ORM führt die Datenbankoperationen aus und Drizzle Kit erzeugt die SQL-Migrationsdateien. Das Anwenden dieser Dateien beim Anwendungsstart übernimmt der Migrator aus `drizzle-orm/libsql/migrator`. Die automatische Anwendung vorbereiteter Migrationen ist somit Bestandteil der eingerichteten Drizzle-Initialisierung und keine alleinige Eigenschaft des Datenbanktreibers. [14]

5.2.4 Erzeugung versionierter Schemamigrationen

Nach der Anpassung der Tabellen wurden zwei Wege zur Einrichtung des Datenbankschemas unterschieden. Mit `drizzle-kit push` können die Schemadefinitionen unmittelbar mit einer angebundenen Datenbank abgeglichen und die Änderungen ohne gesondert gespeicherte SQL-Migrationsdateien angewandt werden. [17] Alternativ erzeugt `drizzle-kit generate` SQL-Dateien anhand der aktuellen Definitionen und der bisherigen Schema-Snapshots. Diese Dateien können anschließend durch den Drizzle-Migrator angewandt werden. [18]

Für den Demonstrator wurde der zweite Weg gewählt. Die Migrationsdateien liegen im Verzeichnis `drizzle` und können gemeinsam mit dem Anwendungscode versioniert und ausgeliefert werden. Beim Start verarbeitet der Migrator die noch nicht angewandten Migrationen. Damit steht ein nachvollziehbarer Ablauf für spätere Schemaänderungen bereit, ohne die vorhandene Benutzerdatenbank bei jedem Update neu anzulegen. Die Verträglichkeit einer konkreten Schemaänderung mit bestehenden Daten muss dennoch anhand der jeweiligen Migration geprüft werden.

Die resultierende Konfiguration beschreibt deshalb nur den SQLite-Dialekt, den Einstiegspunkt der Schemadefinitionen und das Ausgabeverzeichnis der Migrationen. Ein Verbindungsparameter zur Entwicklungsdatenbank ist für die Generierung nicht mehr erforderlich. [18] Der Laufzeitpfad der Benutzerdatenbank wird separat im Main-Prozess festgelegt. Quellcode 5.3 zeigt diese Konfiguration des Demonstrators. [14]

```ts
import { defineConfig } from 'drizzle-kit'

export default defineConfig({
  schema: './src/main/db/schema/index.ts',
  out: './drizzle',
  dialect: 'sqlite'
})
```

Quellcode 5.3: Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen.

In `package.json` wurden hierfür die Skripte `db:generate` mit dem Befehl `drizzle-kit generate` und `db:check` mit `drizzle-kit check` eingerichtet. Die Anwendung selbst benötigt zur Laufzeit keine Ausführung von Drizzle Kit: Sie verwendet die bereits erzeugten Dateien über den ORM-Migrator. Ein zusätzliches Skript für `push` wäre eine Möglichkeit für die Entwicklung, ist jedoch im aktuellen Zielprojekt nicht eingerichtet. [14]

5.2.5 Datenbankinitialisierung beim Anwendungsstart

Die Initialisierung wurde in den Startablauf des Main-Prozesses eingebunden. Innerhalb des mit `app.whenReady()` begonnenen Ablaufs in `src/main/index.ts` wird `initDatabase()` abgewartet, bevor die lokale Datenbank an Repositories und Dienste übergeben und das Anwendungsfenster geöffnet wird. Scheitert die Vorbereitung, zeigt die Anwendung eine Fehlermeldung an und beendet den Start. Die Oberfläche wird dadurch nicht mit einer unvollständig vorbereiteten Datenbank geöffnet. [14]

Der Speicherort wird in `src/main/db/index.ts` aus `app.getPath('userData')` und dem Unterverzeichnis `data` gebildet. Electron stellt damit einen plattformabhängigen Pfad für die Daten der jeweiligen Anwendung bereit. [15] In der paketierten Anwendung lautet der Dateiname `studyplan.db`, während die Entwicklung `studyplan-dev.db` verwendet. Unter Windows ergibt sich bei der verwendeten Anwendungskonfiguration beispielsweise `%APPDATA%\studyplan\data\studyplan.db`, üblicherweise innerhalb von `AppData\Roaming`. [14] Der Unterordner wird vor dem Öffnen der Datei angelegt; die Datenbank bleibt so von den ausgelieferten Programmdateien getrennt.

Danach öffnet der libSQL-Client die Datei über eine lokale `file:`-URL. Die Initialisierung setzt ausdrücklich `PRAGMA foreign_keys = ON` und `PRAGMA journal_mode = WAL`. Die erste Einstellung aktiviert die Prüfung der im Schema tatsächlich definierten Fremdschlüsselbedingungen; sie ergänzt selbst keine solchen Bedingungen. Die zweite wählt den Write-Ahead-Logging-Modus für die Datenbank. [19] Anschließend werden die ausstehenden Migrationen angewandt und die Verbindung mit einer einfachen Abfrage geprüft. Die konkrete Implementierung legt außerdem die SQLite-kompatible Migrationstabelle `__drizzle_migrations` vor dem Aufruf des Migrators an. [14]

Damit die Migrationen auch in der installierten Anwendung verfügbar sind, wurde das Verzeichnis `drizzle` in `electron-builder.yml` unter `extraResources` aufgenommen. Quellcode 5.4 zeigt diesen Eintrag. In der Entwicklung liest die Anwendung die Dateien aus dem Projektverzeichnis; im paketierten Zustand verwendet sie `process.resourcesPath` und den dort enthaltenen Unterordner `drizzle`. [14]

```yaml
extraResources:
  - from: drizzle
    to: drizzle
    filter:
      - '**/*'
```

Quellcode 5.4: Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt).

Mit diesen Schritten war die lokale Datenhaltung vorbereitet. Dies umfasst die SQLite-Schemadefinitionen, den lokalen Datenbankzugriff sowie die Generierung, Auslieferung und Anwendung der Migrationen. Eine offline nutzbare Kernanwendung folgt daraus erst durch die anschließende Übernahme der fachlichen Operationen und der Oberfläche.

5.2.6 Übernahme des API-Vertrags und Anpassung der Codegenerierung

Nach der Vorbereitung der Datenhaltung wurde die Schnittstelle zwischen Oberfläche und fachlichen Operationen überführt. Als Ausgangspunkt diente der OpenAPI-Vertrag aus `lib/api-spec/openapi.yaml` des Web-Prototyps. Er wurde in das Zielprojekt unter `api/openapi.yaml` übernommen. Die Schreibweise der vorliegenden Dateien lautet somit `.yaml`. Der Vertrag enthält die Operationskennungen, Parameter und Datenschemata, aus denen bereits im Ausgangsartefakt Aufrufhilfen für die Oberfläche und Zod-Schemata erzeugt wurden. [6]

Im Web-Prototyp steuerte `orval.config.ts` die Erzeugung eines React-Query-Clients und der Zod-Schemata durch Orval. Dieses Prinzip der gemeinsamen Vertragsgrundlage wurde übernommen. Im Desktop-Projekt erfolgt die Generierung jedoch durch eigene TypeScript-Skripte unter `scripts/openapi-ipc`. Die Datei `api/ipc-generator.config.ts` wird von diesen Skripten eingelesen; sie ist keine Orval-Konfiguration. Sie legt den Eingabevertrag sowie die Ausgabepfade für gemeinsame API-Module und Renderer-Aufrufhilfen fest. [20]

Für das Einlesen des Vertrags wird das Paket `yaml` verwendet. Zod dient weiterhin zur Beschreibung und Prüfung der Eingabe- und Ausgabedaten. Der Projektgenerator liest die Operationen und Schema-Komponenten aus dem Vertrag und erzeugt daraus TypeScript-Typen, Laufzeitschemata sowie eine Zuordnungstabelle `operationContracts`. Zusätzlich erzeugt er einen transportunabhängigen API-Client und React-Query-Hooks für den Renderer. Die Typen und Zod-Schemata liegen gemeinsam unter `src/shared/api/generated`; die Hooks liegen unter `src/renderer/src/api/generated`. Die Validierung verwendet der Main-Prozess, auch wenn die generierten Module nicht ausschließlich in seinem Quellcodeverzeichnis abgelegt sind. [20]

Die Generierung wird über `api:generate` in `package.json` aufgerufen. Mit `api:check` kann geprüft werden, ob die vorhandenen Dateien der aktuellen Generatorausgabe entsprechen. OpenAPI und YAML werden dabei während der Entwicklung eingelesen. Zur Laufzeit verwendet die Anwendung die daraus erzeugten Module; sie interpretiert den YAML-Vertrag nicht bei jedem API-Aufruf erneut. Die Generatoren erzeugen außerdem keine fachlichen Implementierungen. Eine im Vertrag definierte Operation muss gesondert mit einem Handler verbunden werden. [20]

Die Erzeugung der Renderer-Hooks wurde während der anschließenden Oberflächenintegration weiter ausgearbeitet. Im Commit `f03ab7f` wurde `scripts/openapi-ipc/generator-lib.ts` um die Generierung der React-Query-Aufrufhilfen und eines gemeinsamen Exportmoduls unter `src/renderer/src/api/index.ts` erweitert. Der Generator orientiert sich an den bereits verwendeten Aufrufformen: Query-Hooks erhalten die benötigten Pfad- und Query-Parameter, Mutation-Hooks verwenden ein Variablenobjekt mit `data` für den bisherigen Request-Body. Auch Hilfsfunktionen zur Bildung der Query Keys werden erzeugt. Damit können vorhandene Aufrufmuster der Oberfläche weiterverwendet werden, während der Generator ihre Eingaben auf die interne IPC-Struktur abbildet. [20], [21]

Im Zuge der ersten Authentifizierungsimplementierung wurde auch die Anbindung des generierten Clients angepasst. Zunächst stellte Preload den vollständigen API-Client bereit. Im Commit `1c1988d` wurde seine Erzeugung in den Renderer verlagert: Die generierten Hooks verwenden dort `createApiClient` mit der über `window.apiTransport.invoke` bereitgestellten Transportfunktion. Preload vermittelt die IPC-Aufrufe, während der Client im Renderer erfolgreiche Ergebnisse entnimmt und Fehler als `ApiClientError` rekonstruiert. Diese Änderung ist Teil der Schnittstellenintegration und wird deshalb hier zusammen mit der Generatorentwicklung dokumentiert. [22]

5.2.7 Einrichtung des IPC-Transports und der Preload-Schnittstelle

Der generierte API-Client ruft fachliche Operationen über einen abstrahierten Transport auf. Im Web-Prototyp verwendete der erzeugte Client HTTP-Methode, URL und `customFetch`. Im Desktop-Projekt erhält der Client stattdessen eine Funktion vom Typ `ApiTransport`. Diese übermittelt eine `operationId` und deren typisierte Eingabedaten über die Preload-Schnittstelle. Parameter werden dabei in den Gruppen `path`, `query` und `body` zusammengefasst. Die Bezeichnungen bleiben aus dem Web-Vertrag erhalten, bezeichnen im IPC-Aufruf jedoch Felder eines Objekts statt Bestandteile einer HTTP-Anfrage. [6], [20]

In `src/preload/index.ts` wurde hierfür ein Objekt `apiTransport` eingerichtet. Dessen Methode `invoke` sendet die Anfrage über `ipcRenderer.invoke` auf den festgelegten Kanal `api:invoke` an den Main-Prozess. Die Anfrage enthält die Operationskennung, das Eingabeobjekt und eine `requestId` zur Zuordnung des Ergebnisses. Der Renderer erhält über `contextBridge.exposeInMainWorld` Zugriff auf `window.apiTransport`. Dieses Zusammenspiel von Preload-Bridge, `ipcRenderer.invoke` und `ipcMain.handle` entspricht dem von Electron dokumentierten Muster für einen Aufruf mit Rückgabewert. [20], [23]

Ergänzend wurde in `src/preload/index.d.ts` das globale TypeScript-Interface `Window` um die typisierte Transportmethode erweitert. Diese Deklaration beschreibt die Schnittstelle für die Typprüfung; das tatsächlich verfügbare Objekt wird durch den Preload-Code bereitgestellt. Der Renderer verwendet dadurch die vorgesehenen API-Aufrufe, ohne einen frei wählbaren IPC-Kanal oder das gesamte `ipcRenderer`-Objekt zu erhalten. Die Electron-Dokumentation empfiehlt eine begrenzte Bridge anstelle der Weitergabe des vollständigen IPC-Zugriffs. [24]

Ein IPC-Handler für diesen Aufrufweg wird im Main-Prozess registriert. Im Renderer befinden sich die Aufrufhilfen und Hooks; dort wird kein `ipcMain.handle` eingerichtet und kein Main-IPC-Ereignis verarbeitet. Die Übernahme der React-Komponenten und die dafür erforderlichen Anpassungen

ihrer API-Nutzung werden im anschließenden Entwicklungsschritt behandelt. Die hier beschriebene Schnittstelle schafft zunächst die technische Grundlage dafür. [20]

5.2.8 Validierung und Zuordnung der Operationen im Main-Prozess

Für die zentrale Verarbeitung wurde `src/main/ipc/api-dispatcher.ts` angelegt. Der Dispatcher nimmt die über `api:invoke` übermittelte Anfrage entgegen und ordnet sie anhand der `operationId` einem Vertrag und einem registrierten Handler zu. Im vorliegenden Implementierungsstand prüft er zunächst den zulässigen Absender sowie die grundlegende Form der Anfrage. Anschließend prüft er die Eingabedaten mit dem zur Operation erzeugten Zod-Schema. Unbekannte Operationskennungen und ungültige Eingaben werden abgewiesen. [20]

Nach der Ausführung des Handlers validiert der Dispatcher auch dessen Rückgabewert. Für ausdrücklich deklarierte Fehlerantworten verwendet er ebenfalls die erzeugten Fehlerschemata. Das Ergebnis wird in einem gemeinsamen Format zurückgegeben: Ein erfolgreicher Aufruf enthält `ok: true`, die Daten und eine Anfragekennung; ein fehlgeschlagener Aufruf enthält `ok: false` und ein Fehlerobjekt. Der API-Client entnimmt einem erfolgreichen Ergebnis die Daten und überführt einen Fehler in `ApiClientError`. Damit wird die bisherige HTTP-Antwortbehandlung durch eine ausdrückliche IPC-Ergebnisbehandlung ersetzt. [20]

Die Handler-Zuordnung wird unabhängig von der Generierung eingerichtet. Der Dispatcher implementiert selbst weder die fachliche Auswahl noch den Datenbankzugriff. Bei einer bekannten, aber nicht registrierten Operation liefert er `NOT_IMPLEMENTED`. Die Generierung eines Vertrags oder Hooks belegt daher noch nicht die Ausführbarkeit der zugehörigen Funktion. Fachliche Berechtigungen und die Zuordnung einer lokalen Sitzung werden zusätzlich in den jeweiligen Handlern beziehungsweise Diensten behandelt. Die detaillierte Härtungsprüfung dieser Schnittstellen folgt in Phase 3. [20]

5.2.9 Begründung und Grenzen der vertragsbasierten Generierung

Eine mögliche Alternative bestand darin, die von Orval erzeugten Hooks, Typen und Zod-Schemata manuell zu übernehmen und an IPC anzupassen. Damit wäre die Entwicklung eines eigenen Generators entfallen. Die Schnittstellendefinitionen hätten jedoch weiterhin in den Aufrufhilfen und Validierungsschemata gepflegt und bei Änderungen miteinander abgestimmt werden müssen. Für den angestrebten erweiterbaren Demonstrator wurde deshalb die gemeinsame Vertragsgrundlage beibehalten und ihre Übersetzung in den IPC-Aufrufweg automatisiert.

Der Vertrag legt fest, welche Operationen verfügbar sind und welche Eingaben, Rückgabewerte und deklarierten Fehler sie besitzen. Aus dieser gemeinsamen Beschreibung werden die Typdefinitionen, Client-Aufrufe, Hooks und Zod-Schemata abgeleitet. Eine Änderung der Schnittstellenbeschreibung wird dadurch an einer zentralen Stelle vorgenommen und anschließend durch erneute Generierung in die abhängigen Module übertragen. Für zusätzliche Operationen ergibt sich ein wiederholbares Vorgehen: den Vertrag ergänzen, die Schnittstellenmodule erzeugen und die zugehörigen Handler sowie ihre fachliche Verarbeitung implementieren. Oberflächenabläufe, Geschäftslogik und Berechtigungsprüfungen bleiben dabei gesonderte Entwicklungsaufgaben. [20]

Die eigenen Generator-Skripte bündeln die Regeln zur Abbildung des übernommenen Vertrags auf Operationskennungen, strukturierte IPC-Eingaben und die gemeinsame Ergebnisbehandlung. Diese Regeln werden damit für alle eingeschlossenen Operationen nach demselben Verfahren angewendet. Die Entscheidung verbindet die Wiederverwendung des Ausgangsvertrags nach A3 mit der Erweiterbarkeit nach A8 und einem nachvollziehbaren Vorgehen nach A9. Sie begründet sich durch die konsistente Pflege und wiederholbare Transformation der Schnittstellen; ein geringerer Gesamtaufwand gegenüber einer manuellen Übernahme wurde nicht quantitativ nachgewiesen.

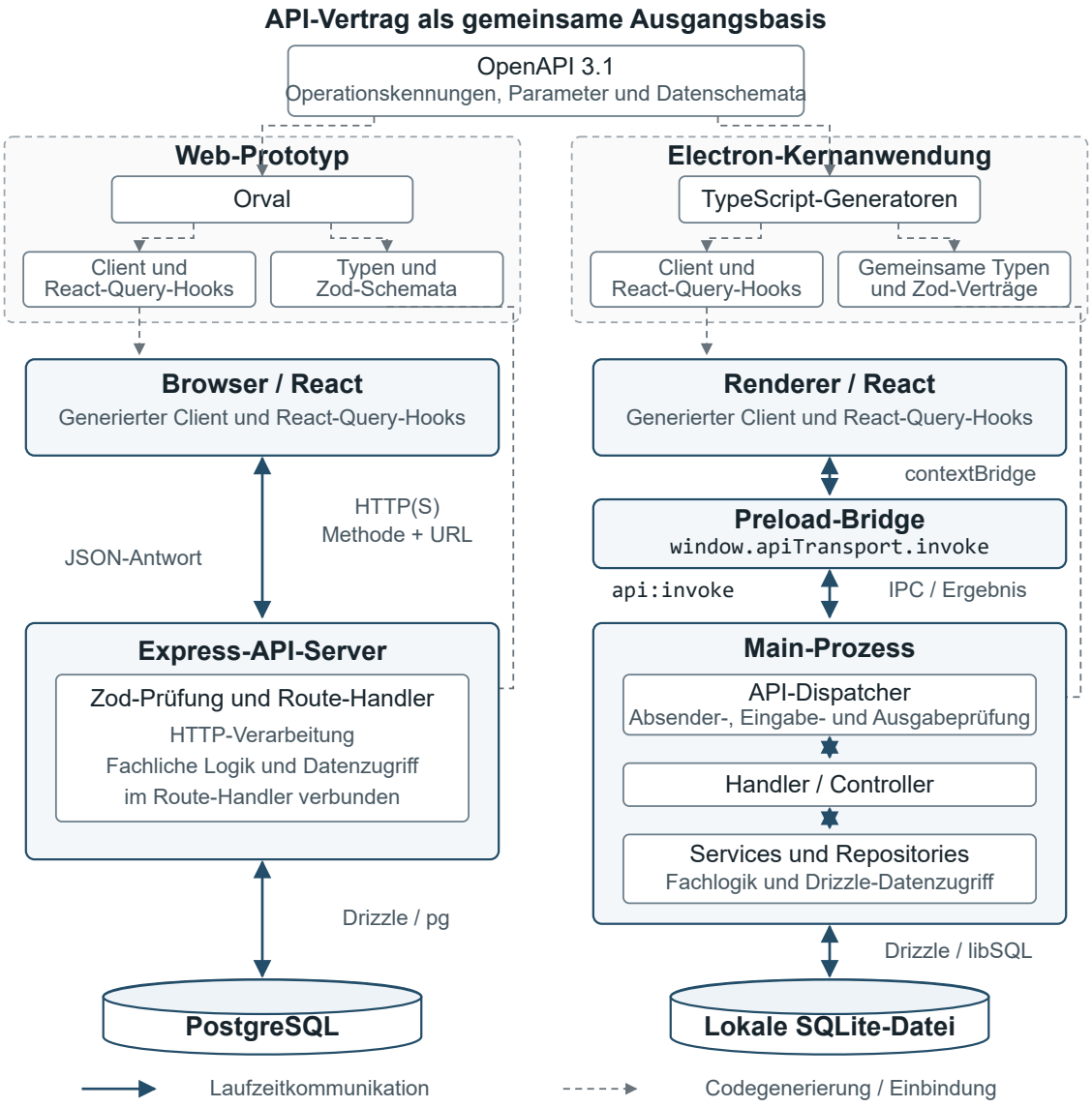


Abbildung 5.1: Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt.

Das Architekturziel ist eine verbindliche, gemeinsam definierte und typisierte Schnittstelle zwischen Renderer und Main-Prozess. Beide Seiten hängen von diesem Vertrag ab. Die fachliche Implementierung bleibt im Main-Prozess und wird vom Renderer über die festgelegten Aufrufe angesprochen. Diese Bindung an einen gemeinsamen Vertrag unterstützt die Trennung der Verantwortlichkeiten und ihre unabhängige Prüfung im Sinne von A2.

Abbildung 5.1 stellt den ursprünglichen Aufrufweg demjenigen der lokalen Electron-Kernanwendung gegenüber. In beiden Fällen bildet ein gemeinsamer Vertrag die Grundlage für Client und Validierung. Die wesentliche Änderung liegt in der Umsetzung dieses Vertrags zur Laufzeit: HTTP-Aufrufe an einen Express-Server werden durch begrenzte Aufrufe über Preload und IPC ersetzt. Die eingezeichnete Trennung in Handler, Services und Repositories zeigt die resultierende Desktop-Architektur; ihre fachliche Ausarbeitung ist ein weiterer Migrationsschritt. [6], [20]

Die Umsetzung der gemeinsamen Vertragsgrundlage im Web-Prototyp und in der Electron-Kernanwendung wird in Tabelle 5.2 gegenübergestellt.

Aspekt	**Web-Prototyp**	**Electron-Kernanwendung**
Vertragsgrundlage	OpenAPI mit Operationskennungen und Datenschemata.	Bestehende Definitionen übernommen und gezielt ergänzt.
Generierung	Orval für React Query und Zod.	Projektspezifische TypeScript-Generatoren für Client, Hooks und gemeinsame Zod-Verträge.
Aufrufweg	HTTP-Methode und URL über `customFetch`.	`operationId` und Eingabeobjekt über Preload und den Kanal `api:invoke`.
Parameter	Pfad, Query-String und HTTP-Body.	Strukturierte Eingabe mit `path`, `query` und `body`.
Ergebnis	HTTP-Antwort beziehungsweise HTTP-Fehler.	Geprüftes `ApiResult`; Fehler werden als `ApiClientError` weitergegeben.
Datumswerte	Strings im generierten Client; Datumsprüfung in Zod-Schemata.	`Date`-Werte und `z.date()` in den gemeinsamen Modulen.
Zugriffsgrenze	Serverseitige HTTP-Verarbeitung und Sitzungszuordnung.	Begrenzte Bridge, Absender- und Schemaprüfung sowie fachliche Sitzungsprüfung.

Tabelle 5.2: Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung.

OpenAPI ist als Standard zur Beschreibung von HTTP-APIs definiert. [25] Die hier entwickelte Nutzung für IPC ist eine projektspezifische Übersetzung ausgewählter Vertragsbestandteile. Methoden und Pfade bleiben als Metadaten erhalten; die tatsächliche Weiterleitung verwendet die Operationskennung. Der Generator unterstützt die im Projekt benötigten Schemakonstrukte und lokale Referenzen, beansprucht aber keine vollständige Umsetzung sämtlicher OpenAPI-Funktionen. Beispielsweise weist er rekursive Komponentenschemata zurück. Die Wiederverwendung des Vertrags ist deshalb ein belegtes Prinzip dieses Falls und kein Nachweis allgemeiner HTTP- oder OpenAPI-Kompatibilität des IPC-Transports. [20]

5.2.10 Übernahme der React-Oberfläche in den Renderer

Nach der Vorbereitung der Datenhaltung und der Schnittstelle wurde die Oberfläche des Ausgangsartefakts in den Electron-Renderer übernommen. Damit wurde die bei der Projektinitialisierung gewählte Kombination aus React, TypeScript und Vite praktisch aufgegriffen. Als Ausgangspunkt diente `artifacts/studienplaner/`. Die vorhandenen Seiten, das Anwendungslayout, die UI-Komponenten sowie die Hilfsmodule unter `hooks/` und `lib/` wurden in `src/renderer/src/` überführt. Hierzu gehören beispielsweise die Badge-, Formular- und Dialogkomponenten sowie die Seiten für Dashboard, Kalender, Noten und Stundenplan. Die Übernahme ist im Commit `f4ea658` dokumentiert. [21]

Auch `App.tsx` und der React-Einstiegspunkt `main.tsx` wurden übernommen. Die vorhandene Einbindung von React Query über `QueryClientProvider` sowie die Anbieter für Tooltips und Benachrichtigungen blieben als Grundlage der Oberfläche erhalten. Die Darstellungsstruktur konnte dadurch weitgehend als vorhandener React-Code weiterverwendet werden; die folgende Integration betraf insbesondere Modulauflösung, Datenzugriffe und Navigation. [6], [21]

Für die Gestaltung wurde die Web-Stylesheet-Datei `index.css` übernommen und weiterhin in `main.tsx` importiert. Sie enthält die Theme-Variablen sowie die Einbindung von Tailwind CSS, `tw-animate-css` und dem Typografie-Plugin. Die Dateien `assets/main.css` und `assets/base.css` gehörten dagegen zur Electron-Vorlage und wurden zusammen mit deren Beispielkomponenten und Grafiken entfernt. Sie sind somit kein zusätzlich übernommener Bestandteil der Web-Oberfläche. [21]

Beim anfänglichen Kopieren gelangten außerdem die eigenständige `package.json`, `vite.config.ts`, `tsconfig.json`, `components.json` und eine zusätzliche `index.html` in den Renderer-Quellcodeordner. Nach dem Abgleich mit der Electron-Projektstruktur wurden diese Dateien im Commit `8b9bd66` entfernt. Der HTML-Einstiegspunkt liegt in der Zielanwendung unter `src/renderer/index.html`; Paketdefinitionen und Build-Konfiguration werden durch die vorhandenen Dateien des Electron-Projekts verwaltet. Die kopierten Konfigurationen dienten damit als Grundlage für den Abgleich, wurden aber nicht als zweites Frontend-Projekt weiterbetrieben. [21]

5.2.11 Abgleich von Build-Konfiguration und Frontend-Abhängigkeiten

Der Konfigurationsabgleich unterschied die für die Darstellung erforderlichen Einstellungen von der ursprünglichen Web-Bereitstellung. Die Web-Vite-Konfiguration enthielt neben React und Tailwind auch Replit-spezifische Entwicklungsplugins, die Umgebungsvariablen `PORT` und `BASE_PATH` sowie Einstellungen für den Webserver und dessen Ausgabeverzeichnis. Diese Bereitstellungsvorgaben wurden für den Electron-Renderer nicht übernommen. [6] electron-vite verwaltet die Konfigurationen für Main, Preload und Renderer in einer gemeinsamen Datei; seine Renderer-Konfiguration verwendet standardmäßig `src/renderer` als Projektwurzel und die dortige `index.html` als Einstiegspunkt. [26]

Für die übernommenen lokalen Imports war der Alias `@` anzupassen. Im Commit `ceccf8f` wurde in `electron.vite.config.ts` unter `renderer.resolve.alias` der Eintrag `'@': resolve('src/renderer/src')` ergänzt. Parallel wurde in `tsconfig.web.json` die Zuordnung `@/*` zu `src/renderer/src/*` eingerichtet. Der bereits vorhandene Alias `@renderer` blieb erhalten. Damit konnten Imports wie `@/components/ui/badge` nach dem Verschieben der Dateien unverändert verwendet werden. [21] Beide Zuordnungen erfüllen unterschiedliche Aufgaben: Vite löst die Module für den Build auf; TypeScript verwendet `paths` für seine Modul- und Typprüfung. Die TypeScript-Einstellung allein schreibt die ausgegebenen Importpfade nicht um. [27]

Für die Verarbeitung der übernommenen Styles wurde im Renderer-Bereich von `electron.vite.config.ts` das Plugin `@tailwindcss/vite` importiert und mit `tailwindcss()` neben `react()` registriert. Diese Anpassung befindet sich ebenfalls im Commit `f03ab7f`, dient jedoch der CSS-Verarbeitung und ist von den dort vorgenommenen Generatoränderungen zu unterscheiden. [21] Die Kombination aus Vite-Plugin und Tailwind-Import in der CSS-Datei entspricht dem dokumentierten Integrationsweg für Tailwind CSS. [28] Zusätzlich wurde die Vite-Typreferenz in `src/renderer/src/env.d.ts` wieder eingerichtet. [21]

Die benötigten Frontend-Pakete wurden anhand der Paketdefinition des Ausgangsartefakts und seiner Abhängigkeitsauflösung in das Electron-Projekt aufgenommen. Dazu zählen die Radix-UI-Pakete der übernommenen shadcn/ui-Komponenten, `clsx`, `class-variance-authority` und `tailwind-merge` für die Gestaltung sowie React Hook Form, dessen Resolver, React Query, `date-fns`, Lucide und Wouter. Die shadcn/ui-Komponenten liegen dabei als übernommene Quelldateien im Projekt vor. Die Paketdefinition des Electron-Projekts wurde ergänzt; die kopierte Frontend-`package.json` konnte anschließend entfallen. Das Ausgangsartefakt verwendet `pnpm-lock.yaml`, während das Zielprojekt seine installierten Abhängigkeiten in `package-lock.json` festhält. Die Abhängigkeitsauflösung wurde daher im Zielprojekt neu erstellt. [6], [21]

5.2.12 Anbindung der übernommenen Seiten und Anpassung der Datumswerte

Die bisherigen Imports aus `@workspace/api-client-react` wurden in den Seiten und im Layout durch Imports aus `@/api` ersetzt. Dieses Modul exportiert die für IPC erzeugten Hooks und zugehörigen Typen. Die Generatoranpassungen aus Abschnitt 5.2.6 ermöglichten, vorhandene Hook-Namen und typische Aufrufmuster beizubehalten. Beispielsweise blieb die Verwendung von `useListEvents(monthParams)` beziehungsweise einer Mutation mit `{ data: ... }` erhalten. Die Änderung des Imports allein genügte allerdings dort nicht, wo sich die Datendarstellung an der Schnittstelle änderte. [21]

Dies zeigt der Commit `ce275e4` an `src/renderer/src/pages/kalender.tsx`. Während der Web-Client Datumswerte als Strings bereitstellte, verwenden die generierten Desktop-Typen und Zod-Schemata `Date`-Werte. In `eventsForDay` wurde deshalb `parseISO(e.startDate)` durch `new Date(e.startDate)` ersetzt; dieselbe Anpassung erfolgte für das Enddatum. Bei der Formularübermittlung werden die eingegebenen Datumsstrings mit `new Date(...)` in die erwarteten Werte umgewandelt. Abfragegrenzen werden als `Date` übergeben, und bereits vorhandene Datumswerte können unmittelbar formatiert oder über `getTime()` sortiert werden. Das Dashboard wurde entsprechend angepasst. Die Änderungen betreffen damit die Schnittstellenwerte und deren Verarbeitung, während die bestehende Darstellungsstruktur erhalten bleibt. [21]

Im selben Arbeitsschritt wurden ungenutzte Imports, Variablen und Zustandseinträge entfernt. Diese Bereinigung änderte den fachlichen Ablauf der betreffenden Seiten nicht. Sie war jedoch auch für die Typprüfung relevant: Die ursprüngliche TypeScript-Basiskonfiguration setzte `noUnusedLocals` auf `false`, während die vom Electron-Projekt geerbte Toolkit-Konfiguration `noUnusedLocals` und `noUnusedParameters` aktiviert. [6], [21] TypeScript meldet bei aktiviertem `noUnusedLocals` ungenutzte lokale Deklarationen als Fehler. [29] Die Bereinigung ist daher im untersuchten Projekt nicht ausschließlich eine Frage des Programmierstils.

Noch nicht überführt waren damit alle Datenzugriffe der Oberfläche. Insbesondere enthielten die kopierten Kalender- und Notenseiten direkte HTTP-Aufrufe für Dateiimporte, die nicht über die generierten Hooks liefen. Ihre Anpassung wird bei der Umsetzung der entsprechenden fachlichen Operationen behandelt. Aus der Übernahme der Seiten und ihrer Hook-Anbindung folgt somit noch keine vollständig offline nutzbare Kernanwendung. [6], [21]

5.2.13 Anpassung der Navigation an lokale Renderer-Ressourcen

Für die Navigation wurde das bereits verwendete Paket `wouter` beibehalten. Im Web-Prototyp erhielt dessen Router den aus `import.meta.env.BASE_URL` abgeleiteten Basispfad. Im Authentifizierungscommit `1c1988d` wurde stattdessen `useHashLocation` aus `wouter/use-hash-location` eingebunden und mit `<WouterRouter hook={useHashLocation}>` aktiviert. [21] Der Hook verwendet den Teil der Adresse hinter `#` als Anwendungsroute. [30] Eine Seite wie `/kalender` kann dadurch innerhalb der geladenen Renderer-Ressource als `#/kalender` adressiert werden. Der Pfad der lokalen HTML-Datei bleibt bestehen, während Wouter die Seitenauswahl übernimmt. Die vorhandenen React-Routen und Navigationskomponenten konnten auf dieser Grundlage weiterverwendet werden. [21]

5.2.14 Lokale Authentifizierung als erster vollständiger Verarbeitungspfad

Nach der Übernahme der Oberfläche begrenzte der vorhandene Anmeldeablauf zunächst den Zugang zu den fachlichen Seiten. Das Layout ruft `useGetMe` auf und setzt einen zugeordneten Benutzer voraus. Für die weitere Entwicklung wurde die Anmeldung deshalb beibehalten und als erste Handlergruppe implementiert. Damit konnte der bereits vorbereitete Aufrufweg zwischen Oberfläche, Preload, Dispatcher und lokaler Datenhaltung an einem zusammenhängenden Ablauf genutzt werden. Dieser Schritt ist im Commit `1c1988d` dokumentiert. [22]

Die Umsetzung gliedert die Verantwortlichkeiten in drei Teile. `src/main/ipc/handlers/auth.handlers.ts` ordnet die Operationen `register`, `login`, `logout` und `getMe` den fachlichen Aufrufen zu und übersetzt deklarierte Fehler in das gemeinsame API-Fehlerformat. `src/main/services/auth.service.ts` implementiert Registrierung, Passwortprüfung und Benutzerzuordnung. `src/main/repositories/users.repository.ts` übernimmt die Benutzerabfragen und das Anlegen der Datensätze über Drizzle. Beim Anwendungsstart werden Repository und Dienste nach der Datenbankinitialisierung erzeugt und ihre Handler beim Dispatcher registriert. Damit wird die in A2 verlangte Trennung von Schnittstellenverarbeitung, Fachlogik und Datenzugriff erstmals für eine vollständige Operationsgruppe umgesetzt. [22]

Für das Hashen und Vergleichen der Passwörter wurde das bereits im Web-Backend verwendete Paket `bcryptjs` mit dem dort angegebenen Versionsbereich `^3.0.3` übernommen. Passwort-Hashes bleiben im Main-Prozess und in der lokalen Benutzertabelle; die Oberfläche erhält nur die vorgesehenen Benutzerdaten. An die Stelle der HTTP-Sitzung tritt `SessionService`, der die aktuelle Benutzerkennung im Main-Prozess hält. Diese lokale Sitzung liegt im Arbeitsspeicher und endet beim Beenden der Anwendung; die Benutzerdatensätze bleiben dagegen in der SQLite-Datenbank gespeichert. Die lokale Anmeldung ist von der später ergänzten Authentifizierung gegenüber entfernten Kollaborationsdiensten zu unterscheiden. [6], [22]

Im Renderer wurden erfolgreiche Anmeldungen und Registrierungen mit dem React-Query-Cache verbunden: Der bisherige Cache wird geleert und die zurückgegebene Benutzerinformation unter dem Query Key von `getMe` gespeichert. Anschließend erfolgt die Navigation zur Anwendung. Der Fehlercode `UNAUTHENTICATED` führt im Layout zur Anmeldung und wird von der Wiederholungslogik der Queries ausdrücklich berücksichtigt; andere Ladefehler erhalten eine gesonderte Anzeige mit erneuter Abfragemöglichkeit. Auch die Weiterleitung nach einer bereits bestehenden Anmeldung wurde in einen React-Effekt verlagert. Diese Änderungen ergänzen die Übernahme der Oberfläche um die Behandlung des lokalen Sitzungs- und Fehlerzustands. [22]

5.2.15 Integration der CRUD-Funktionen am Beispiel der Notenverwaltung

Nach der Oberflächenintegration und der lokalen Authentifizierung wurden die weiteren fachlichen Funktionen mit dem vorbereiteten Aufrufweg verbunden. Die Migration der CRUD-Operationen wird exemplarisch an der Notenverwaltung beschrieben. Anlegen, Lesen, Bearbeiten und Löschen folgen in den persönlichen Anwendungsbereichen einer wiederkehrenden Grundstruktur; eine separate Beschreibung jeder Variante würde überwiegend dieselben Transformationsschritte wiederholen. Die Notenverwaltung wurde im Commit `f97642b` integriert, zunächst ohne CSV-Import. Als fachliche Vorlage dienten die ursprünglichen Routen aus `artifacts/api-server/src/routes/grades.ts`. [6], [31]

Die Übertragung bestand darin, die innerhalb der Route verbundenen Aufgaben nach ihren Verantwortlichkeiten aufzuteilen. HTTP-spezifische Zugriffe auf `req` und `res` entfielen. Die Eingabeprüfung wurde durch die bereits eingerichtete Vertragsvalidierung des Dispatchers übernommen. Die Verarbeitung der Operationen und ihrer Fehler wurde den Handlern zugeordnet, die fachlichen Regeln dem Service und die Drizzle-Abfragen dem Repository. Tabelle 5.3 zeigt diese Zuordnung am untersuchten Beispiel. [31]

Schicht	Zielmodul	Aufgabe im Notenbeispiel
Schnittstelle	`grades.handlers.ts`	Operationszuordnung, Übernahme der Sitzungskennung und Übersetzung fachlicher Fehler.
Fachlogik	`GradesService`	Anlegen und Ändern fachlicher Werte, Notenstatistik und Veranlassung von Aktivitätseinträgen.
Datenzugriff	`GradesRepository`	Lesen und Schreiben der Notendatensätze über Drizzle mit benutzerbezogenen Auswahlbedingungen.

Tabelle 5.3: Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository.

Die Module wurden in `src/main/index.ts` in den Startablauf eingebunden. Nach der Datenbankinitialisierung werden `GradesRepository` und `ActivityLogRepository` mit der Datenbankverbindung erzeugt. Beide werden an `GradesService` übergeben. `createGradesHandlers` verbindet anschließend die Service-Aufrufe mit den vorgesehenen Operationskennungen und der bereits vorhandenen `SessionService`-Instanz. Die resultierenden Handler werden in die gemeinsame Registry aufgenommen, die der IPC-Dispatcher verwendet. Die fachliche Implementierung wird damit an die zuvor erzeugten Schnittstellen angeschlossen. [31]

Die Benutzerkennung stammt bei diesen Operationen aus der lokalen Sitzung im Main-Prozess. Der gemeinsame Wrapper `authenticated` stellt sie dem Handler bereit. Im Repository werden Änderungen und Löschungen anhand der Datensatzkennung und der Benutzerkennung eingeschränkt. Findet eine solche Operation keinen passenden Datensatz, meldet der Service einen fachlichen Fehler, den der Handler in `NOT_FOUND` überführt. Damit bleibt die persönliche Datenzuordnung Bestandteil des vollständigen Verarbeitungspfads. [31]

Die vorhandenen fachlichen Regeln wurden als Ausgangspunkt beibehalten. Dazu gehören optionale Werte, die Erfassung von Aktivitäten, der ECTS-gewichtete Notendurchschnitt und die Auswertung nach Semester. Anpassungen ergaben sich aus der lokalen Datenrepräsentation: Note und ECTS werden im SQLite-Schema als numerische Werte verarbeitet, sodass die bisher für PostgreSQL verwendete Umwandlung in Strings beziehungsweise deren Rückkonvertierung entfallen konnte. Die Migration übernahm somit die fachlichen Berechnungen und gliederte ihre Ausführung neu. [6], [31]

5.2.16 Wiederkehrender Verarbeitungspfad und fachliche Prüfung

Für die integrierten CRUD-Funktionen ergibt sich der Aufrufweg **Formular → Hook → Preload/ IPC → Dispatcher → Handler → Service → Repository → SQLite**. Bei der Notenerfassung übermittelt die vorhandene Seite beispielsweise die Formulardaten über `useCreateGrade` beziehungsweise `useUpdateGrade`. Der generierte Hook bildet die Eingabe auf den IPC-Vertrag ab. Nach Prüfung und Ausführung gelangt das Ergebnis über den Dispatcher und den API-Client zurück zum Hook. Die Oberfläche aktualisiert anschließend die Abfragen für Notenliste und Statistik. [31]

Diese Grundstruktur kann für weitere CRUD-Bereiche erneut angewandt werden. Dabei sind jeweils die zugehörigen Vertragsdefinitionen, fachlichen Regeln, Datenzugriffe und Berechtigungen zu berücksichtigen. Die Wiederholung betrifft die Verteilung der Verantwortlichkeiten und den Kommunikationsweg; die Berechtigungsregeln gemeinsamer Gruppenressourcen sind gesondert von der persönlichen Datenzuordnung zu prüfen. Die Struktur konkretisiert damit A2 und das in A8 geforderte Erweiterungsmuster.

Mit der Notenimplementierung wurde `scripts/grades/grades.test.ts` ergänzt. Der Testaufbau verbindet den generierten API-Client mit Dispatcher, Handlern, Service und Repositories sowie einer SQLite-Datenbank im Arbeitsspeicher. Geprüft werden unter anderem Sitzungsanforderungen, benutzerbezogene Auswahl, Änderungen und Aktivitätseinträge, Statistikberechnung sowie der Umgang mit fremden Datensätzen. Der Transport ruft den Dispatcher im Test direkt auf und ersetzt damit die native IPC-Kommunikation. Die Tests prüfen folglich den fachlichen Verarbeitungspfad und seine Datenzugriffe; sie sind keine automatisierten Oberflächen- oder Electron-Laufzeittests. [31]

5.2.17 Migration des iCal-Dateiimports

Dateiimporte ergänzen die CRUD-Operationen um eine weitere Interaktionsform. Ihr Migrationsmuster wird anhand des iCal-Imports im Commit `fe08220` beschrieben. Im Web-Prototyp wählte die Kalenderseite eine Datei über ein HTML-Dateifeld aus und übermittelte sie als `FormData` mit `fetch` an `/api/events/import-ical`. Im Express-Backend nahm Multer den Upload entgegen. Der Route-Handler las den Inhalt, interpretierte die Kalenderdaten und schrieb die daraus abgeleiteten Termine unmittelbar in die Datenbank. [6], [32]

In der Desktop-Anwendung wurde diese Upload-Verarbeitung in eine lokale Importoperation überführt. Hierfür wurde der OpenAPI-Vertrag um `importEventsIcal` und das Ergebnisschema `EventIcalImportResult` ergänzt. Anschließend wurden Client, Hook und Validierungsschemata neu erzeugt. Die Operation besitzt keine Dateipfad- oder Inhaltsparameter: Der Renderer löst über `useImportEventsIcal` die Auswahl aus, während der Main-Prozess die Datei ermittelt und verarbeitet. Damit wird die in Abschnitt 5.2.6 beschriebene zentrale Vertragspflege an einer zusätzlichen Schnittstelle angewandt. [32]

Die Dateiauswahl übernimmt `NativeIcalFilePicker` im Main-Prozess. Er verwendet `dialog.showOpenDialog` mit einem Filter für ICS-Dateien und bindet den Dialog, sofern verfügbar, an das aus dem IPC-Absender ermittelte Anwendungsfenster. [32] Electron liefert aus diesem Dialog die ausgewählten Pfade beziehungsweise einen Abbruchzustand zurück. [33] Der Pfad bleibt innerhalb der nativen Auswahl und wird nicht vom Renderer als frei wählbares Argument übergeben. Die Implementierung prüft die Endung `.ics`, den regulären Dateityp und eine Größenobergrenze von 10 MiB vor sowie nach dem Einlesen. Anschließend stellt sie den Inhalt als UTF-8-Text für die Verarbeitung bereit. [32]

Die Interpretation der Kalenderdaten wurde in `src/main/services/events-ical.parser.ts` ausgelagert. Das Modul verwendet weiterhin `node-ical`, prüft die grundlegende Kalenderstruktur und bildet verwertbare `VEVENT`-Einträge auf interne Termindaten ab. Fehlende Titel oder ungültige Datumswerte führen zum Überspringen der betreffenden Einträge; diese werden gezählt. Aus der ursprünglichen Verarbeitung wurden unter anderem die Ableitung des Termintyps aus dem Titel und eine Standarddauer bei fehlendem Enddatum übernommen. Der Parser erzeugt die Termindaten ohne Benutzerkennung und ohne Datenbankzugriff. [6], [32]

`EventsService.importEventsIcal` übernimmt die fachliche Verarbeitung der Parserausgabe. Der Service ergänzt die aus der Sitzung stammende Benutzerkennung, übergibt die Datensätze an `EventsRepository.createManyEvents` und veranlasst den Aktivitätseintrag zum Import. Dateiauswahl, Formatinterpretation, fachliche Zuordnung und Datenzugriff erhalten dadurch getrennte Zuständigkeiten. Beim Start wird der native Picker als Abhängigkeit an `createEventsHandlers` übergeben. Die Schnittstelle `IcalFilePicker` ermöglicht zugleich, die Dateiauswahl in Tests durch eine geeignete Implementierung zu ersetzen. [32]

5.2.18 Ergebnisbehandlung und Prüfung des Dateiimports

Für den Import wurde eine ausdrückliche Ergebnisbehandlung eingerichtet. Ein erfolgreicher Import liefert `status: imported` sowie `importedCount` und `skippedCount`. Ein abgebrochener Dateidialog liefert `status: cancelled` mit beiden Zählwerten `0` und verändert den Datenbestand nicht. Ungültige Dateiauswahl oder Kalenderinhalte werden im Handler über die vorgesehenen Fehlerklassen in `VALIDATION_ERROR` und die deklarierte Fehlerantwort überführt. Diese Ergebnisse durchlaufen denselben Dispatcher und API-Client wie die CRUD-Operationen. [32]

In der Kalenderseite ersetzt der generierte Mutation-Hook den direkten HTTP-Aufruf und das HTML-Dateifeld. Während der Operation wird der Importknopf über `isPending` deaktiviert. Nach einem erfolgreichen Import wird die Terminanfrage invalidiert; die Oberfläche zeigt die Anzahl importierter und gegebenenfalls übersprungener Einträge an. Ein Abbruch beendet den Ablauf ohne Erfolgsmeldung, während ein Fehler gesondert angezeigt wird. Die ursprüngliche Upload-Oberfläche wird damit zu einem Auslöser für die im Main-Prozess ausgeführte lokale Importfunktion. [32]

Die vorhandenen Termintests wurden um Parserfälle, den Import für den angemeldeten Benutzer, den Aktivitätseintrag, den Dialogabbruch und ungültige Kalenderinhalte ergänzt. Eine Testimplementierung von `IcalFilePicker` liefert dabei vorgegebene Inhalte oder einen Abbruchzustand. Dadurch lassen sich Parser, Handler, Service und Repository unabhängig von der nativen Dialogoberfläche prüfen. Die tatsächliche Dialoginteraktion und die Prüfungen beim Lesen einer realen Datei werden durch diesen Testaufbau nicht ausgeführt; ihre Bewertung ist vom fachlichen Importtest zu unterscheiden. [32]

5.2.19 CSV als ergänzende Anwendung des Importmusters

Der CSV-Notenimport aus Commit `d37f1b5` verwendet dieselbe Aufteilung in nativen Dateipicker, Import-Handler, Service und Repository. Die Formatverarbeitung übernimmt hier der eigene Parser `grades-csv.parser.ts`, der Spalten erkennt und unter anderem Trennzeichen, zitierte Felder und Dezimalkommas verarbeitet. Die Vertragsoperation `importGradesCsv` und der erzeugte Hook liefern entsprechend den Import- oder Abbruchstatus sowie Zählwerte. CSV zeigt damit, dass das beschriebene Importmuster sowohl eine eingebundene Parserbibliothek als auch einen eigenen Parser aufnehmen kann. [34]

5.2.20 Einordnung der lokalen Funktionsmigration

Die Notenverwaltung und die Dateiimporte zeigen zwei vollständige lokale Verarbeitungspfade auf Grundlage der übernommenen Oberfläche. Weitere persönliche CRUD-Funktionen verwenden dieselbe Verteilung auf Handler, Services und Repositories; die Dateiimporte ergänzen sie um begrenzte native Dateiauswahl und gesonderte Formatverarbeitung. Die Beispiele erläutern damit wiederkehrende Transformationsschritte, ohne jede Funktion einzeln zu beschreiben. Für den Zielzustand der ersten Phase bleibt der Kontrollpunkt auf den gesamten eingeschlossenen Kernumfang zu beziehen. Die resultierende Struktur des Demonstrators ist in seiner technischen Architekturdokumentation beschrieben. [35]

Der Kontrollpunkt dieser Phase betrifft A1, A2, A4 und A5: Die Anwendung muss lokal starten, die vorgesehenen Kernabläufe müssen ohne Netzwerkverbindung funktionieren und Änderungen müssen persistent sein. Für A3 und A8 wird zugleich festgehalten, welche Komponenten weiterverwendet wurden und welche Verhaltensänderungen durch die Transformation entstanden sind.

5.3 Phase 2: Von der Offline-Kernanwendung zur Local-First-Kollaboration

Die zweite Phase ergänzt gezielt Funktionen, die eine Verbindung zu entfernten Diensten benötigen. Die lokale Kernanwendung bleibt dabei die Grundlage. Local-First bedeutet für den hier untersuchten

Aufgabenbereich, dass eine Änderung zunächst lokal verarbeitet werden kann und ihre Übermittlung als nachgelagerter Vorgang erfolgt. Die Phase verfolgt keinen Anspruch auf vollständige Umsetzung sämtlicher Local-First-Eigenschaften.

Im Demonstrator besitzt der synchronisierte Gruppenbereich eine eigene lokale PowerSync-Datenbank. Diese ist von der Datenbank für persönliche Daten und ursprüngliche lokale Gruppen getrennt. Aufgabenänderungen werden in dieser lokalen Datenbank gespeichert und anschließend über den Connector an den entfernten Dienst übermittelt. Für die Zusammenarbeit werden Supabase und PowerSync eingesetzt. Die vorhandenen lokalen Gruppen werden nicht automatisch in den synchronisierten Bereich überführt.

Nicht jede Operation dieses Bereichs ist offline ausführbar. Gruppenanlage, Profilsuche und Einladungsoperationen benötigen im vorhandenen Entwurf eine Online-Anfrage. Auch die Voraussetzungen der Anmeldung sind zu berücksichtigen: Die entfernte Sitzung wird im Demonstrator im Arbeitsspeicher gehalten, sodass nach einem Neustart eine erneute Anmeldung erforderlich ist. [35] Aus der lokalen Speicherung von Aufgaben folgt daher keine uneingeschränkte Offline-Verfügbarkeit des gesamten Kollaborationsbereichs.

Zusätzlich zur Synchronisierung werden weitere Online-Anbindungen separat behandelt. Beim Studiengangkatalog ruft der Main-Prozess externe Daten ab, validiert deren Felder und übernimmt sie in die lokale Datenbank. Scheitert die Aktualisierung, bleiben bereits lokal vorhandene Katalogdaten verfügbar. [36] Dieser Ablauf ist ein Download mit lokalem Datenbestand und keine Synchronisierung gemeinsam bearbeiteter Aufgaben.

Der Kontrollpunkt dieser Phase betrifft insbesondere A6 und A8. Für Aufgaben sind der lokale Abschluss einer Änderung und ihre spätere Übernahme im Kollaborationsablauf getrennt zu beobachten. Bei unmittelbar netzwerkabhängigen Operationen sind Fehlermeldung und Verhalten ohne Verbindung auszuweisen. Bereits lokal verfügbare Funktionen dürfen durch den Ausfall einer zusätzlichen Online-Anbindung nicht unnötig blockiert werden.

5.4 Phase 3: Prüfung und Härtung der vervollständigten Anwendung

Nach der funktionalen Umsetzung wird die Gesamtheit der entstandenen Schnittstellen geprüft. Dazu gehören IPC-Eingänge, lokale Ressourcen, Navigationsmöglichkeiten und externe Kommunikationspfade. Die Electron-Sicherheitscheckliste bildet den festgelegten Bezugsrahmen. [3] Für jede Empfehlung werden Anwendbarkeit, erforderliche Maßnahme, konkreter Beleg und verbleibende Nachweisgrenze dokumentiert.

Die im Demonstrator vorgesehenen Maßnahmen betreffen unter anderem deaktivierte Node-Integration im Renderer, Context Isolation, Renderer-Sandboxing, eine restriktive Content Security Policy und die Begrenzung von Navigation sowie zusätzlichen Fenstern. IPC-Absender und fachliche Eingaben werden im Main-Prozess geprüft. Für externe Dienste sind die zugelassenen Verbindungsziele und ihre Transportkonfiguration zu kontrollieren. Paketbezogene Einstellungen sind am gebauten Artefakt zu prüfen, sofern sie Bestandteil der jeweiligen Empfehlung sind.

Der Kontrollpunkt ist A7. Ein Konfigurationseintrag, ein Unit-Test und eine beobachtete Laufzeitwirkung liefern unterschiedliche Nachweise und werden entsprechend ausgewiesen. Für SDK-interne Netzwerkpfade wird keine Prüfung behauptet, die tatsächlich nicht durchgeführt wurde. Die konsolidierte Projektmatrix dokumentiert insbesondere die begrenzte Bewertung der PowerSync-Redirectbehandlung; diese Nachweisgrenze ist bei der Ergebnisdarstellung zu übernehmen. [37]

5.5 Ausarbeitung der einzelnen Migrationsschritte

Die Phasenübersicht legt die Struktur des Leitfadens fest. Für die abschließende Ausarbeitung ist jeder Transformationsschritt mit seinem konkreten Ausgangszustand, der vorgenommenen Änderung und dem zugehörigen Beleg zu verbinden. Architekturentscheidungen und Kontrollpunkte müssen auf die Anforderungen A1–A9 zurückgeführt werden können. Eine Phasenbeschreibung allein ersetzt diese Dokumentation noch nicht.

Kapitel 6

Evaluation und Demonstration

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

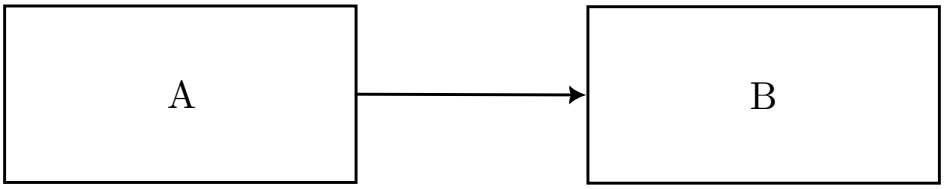


Abbildung 6.1: Example Figure

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Kapitel 7

Diskussion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Kapitel 8

Fazit und Ausblick

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur,

inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine.

Abbildungsverzeichnis

4.1 Ursprüngliche Webarchitektur des Studienplaners mit Laufzeitkommunikation und OpenAPI-basierter Codegenerierung; eigene Darstellung auf Grundlage des Ausgangsartefakts. 12
5.1 Vertragsbasierte Codegenerierung und Aufrufwege im Web-Prototyp und in der lokalen Electron-Kernanwendung. Gestrichelte Pfeile zeigen Generierung oder Einbindung, durchgezogene Pfeile Laufzeitkommunikation. Spätere Synchronisierungs- und Online-Pfade sind nicht dargestellt. 27
6.1 Example Figure 37

Quellcodeverzeichnis

4.1 Exemplarischer Express-Route-Handler für die Abfrage anstehender Termine im Ausgangsartefakt. 13
5.1 Stundenplaneinträge im PostgreSQL-Schema des Ausgangsartefakts (Ausschnitt). 20
5.2 Stundenplaneinträge im SQLite-Schema des Demonstrators (Ausschnitt). 21
5.3 Drizzle-Kit-Konfiguration zur Erzeugung der SQLite-Migrationen. 22
5.4 Auslieferung der Migrationsdateien über zusätzliche Paketressourcen (Ausschnitt). 23

Tabellenverzeichnis

2.1 Example Dataset 5
4.1 Funktionaler Untersuchungsumfang auf Grundlage der ursprünglichen Erstellungs-Prompts. . 10
5.1 Phasen und Kontrollpunkte des fallbezogenen Migrationsleitfadens. 18
5.2 Wiederverwendete Vertragsgrundlage und Änderungen an ihrer Umsetzung. 27
5.3 Aufteilung der ursprünglichen Notenrouten auf Handler, Service und Repository. 31

Glossar

Referenzen

[1] Lovable, „App & Website Builder: Build Software in Minutes“. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://lovable.dev/

[2] K. Peffers, T. Tuunanen, M. A. Rothenberger, und S. Chatterjee, „A Design Science Research Methodology for Information Systems Research“, *Journal of Management Information Systems*, Bd. 24, Nr. 3, S. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[3] Electron, „Security“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/security

[4] Replit, „Web Apps“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.replit.com/features/artifact-types/web-apps

[5] Lovable, „Connect to Supabase“. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://docs.lovable.dev/integrations/supabase

[6] „Studienplaner: Replit-Ausgangsartefakt“. 2026.

[7] M. Kleppmann, A. Wiggins, P. van Hardenberg, und M. McGranaghan, „Local-first software: You own your data, in spite of the cloud“, in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, S. 154–178. doi: 10.1145/3359591.3359737.

[8] Electron, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/process-model

[9] Tauri, „Process Model“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/concept/process-model/

[10] Tauri, „Node.js as a sidecar“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://v2.tauri.app/learn/sidecar-nodejs/

[11] electron-vite, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/guide/

[12] Electron Forge, „React with TypeScript“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/guides/framework-integration/react-with-typescript

[13] Electron Forge, „Getting Started“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronforge.io/

[14] „StudyPlan: Lokale Datenhaltung und Schemamigrationen“. 2026.

[15] Electron, „app: getPath“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/app#appgetpathname

[16] Drizzle Team, „SQLite column types“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/sqlite/column-types

[17] Drizzle Team, „Drizzle Kit: push“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-push

[18] Drizzle Team, „Drizzle Kit: generate“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://orm.drizzle.team/docs/drizzle-kit-generate

[19] SQLite, „Pragma statements supported by SQLite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.sqlite.org/pragma.html

[20] „StudyPlan: Vertragsbasierte API-Generierung und IPC-Schnittstelle“. 2026.

[21] „StudyPlan: Übernahme und Anbindung der React-Oberfläche“. 2026.

[22] „StudyPlan: Lokaler Authentifizierungsablauf“. 2026.

[23] Electron, „Inter-Process Communication“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/tutorial/ipc

[24] Electron, „contextBridge“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/context-bridge

[25] OpenAPI Initiative, „OpenAPI Specification v3.1.0“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://spec.openapis.org/oas/v3.1.0.html

[26] electron-vite, „Configuring electron-vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://electron-vite.org/config/

[27] Microsoft, „TSConfig Reference: paths“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/paths.html

[28] Tailwind Labs, „Installing Tailwind CSS with Vite“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://tailwindcss.com/docs/installation/using-vite

[29] Microsoft, „TSConfig Reference: noUnusedLocals“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.typescriptlang.org/tsconfig/noUnusedLocals.html

[30] Wouter Contributors, „Wouter: Customizing the location hook“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://github.com/molefrog/wouter#customizing-the-location-hook

[31] „StudyPlan: Migration der CRUD-Operationen für Noten“. 2026.

[32] „StudyPlan: Lokaler iCal-Import und seine Ergebnisbehandlung“. 2026.

[33] Electron, „dialog: showOpenDialog“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.electronjs.org/docs/latest/api/dialog#dialogshowopendialogwindow-options

[34] „StudyPlan: Lokaler CSV-Notenimport mit eigenem Parser“. 2026.

[35] „StudyPlan architecture“. 2026.

[36] „Public sample study catalog“. 2026.

[37] „Konsolidierte Electron-Sicherheitsergebnismatrix“. 2026.

Anhang A

Ursprüngliche Erstellungs-Prompts

Dieser Anhang dokumentiert die beiden ursprünglichen Prompts, mit denen der Replit-Webprototyp Studienplaner erstellt und um Studiengruppen erweitert wurde. Die Texte werden im Wortlaut der vom Verfasser bereitgestellten Datei wiedergegeben. Rechtschreibung und Formulierungen bleiben erhalten; Überschriften, Absatzdarstellung und die Trennung der beiden Prompts wurden für die Dokumentation ergänzt.

Die Prompts belegen die Beschreibung von Funktionen und Nutzungsszenarien ohne Festlegung eines UI-Frameworks oder einer konkreten technischen Architektur. Sie enthalten zugleich Anforderungen an Anmeldung, Zugriffsschutz und die Trennung persönlicher und gemeinsamer Inhalte. Der ursprünglich gewünschte Funktionsumfang ist von dem in Kapitel 4 festgelegten Untersuchungsumfang zu unterscheiden; insbesondere begründet der im zweiten Prompt genannte Gerätewechsel keine Erweiterung des Migrations- oder Evaluationsumfangs dieser Arbeit.

Quelle ist die bereitgestellte Textdatei `Pasted-Hi-Ich-m-chte-eine-Web-App-f-r-mein-Studium-bauen-damit_1783021865134.txt`. Eine unveränderte Kopie ist im Thesis-Projekt unter `content/sources/original-creation-prompts.txt` abgelegt. Die Kennzeichnung „Prompt 2“ in der Quelldatei dient hier zur Aufteilung in die folgenden Abschnitte.

A.1 Prompt 1: Persönliche Studienorganisation

> Hi! Ich möchte eine Web-App für mein Studium bauen, damit ich endlich mal mein ganzes Chaos im Griff habe. Ich habe mir das so vorgestellt:
>
> Am Anfang soll man auf ein Dashboard kommen. Das ist quasi die Startseite, wo ich sofort sehe, wie mein aktueller Notenschnitt (GPA) ist, wie viele ECTS-Punkte ich insgesamt schon gesammelt habe und welche Fristen als Nächstes ablaufen. Da sollen auch ein paar schnelle Buttons sein, womit ich sofort eine neue Note oder einen neuen Termin eintragen kann, ohne erst mühsam in die Untermenüs zu klicken. Und eine kleine Liste mit den letzten Aktivitäten und den nächsten Events wäre super.
>
> Damit ich meine Termine im Blick habe, brauche ich einen Kalender. Ganz klassisch als Monatsübersicht. Wenn ich auf einen Tag klickt, soll sich so ein kleines Infofenster (Popover) öffnen, das mir alle Termine für den Tag anzeigt. Und wenn ich einen Termin anklicke, will ich die Details sehen und ihn auch löschen können. Die Termine müssen farblich sortiert sein, damit man sofort sieht, was Sache ist: Prüfungen und Deadlines in Rot, Anmeldefristen in Gelb, normale Vorlesungen in Blau und alles andere in Grün. Ganz wichtig: Ich will meine Termine von der Uni als iCal- oder ICS-Datei hochladen können, damit ich nicht alles einzeln abtippen

muss! Manchmal gehen Events auch über mehrere Tage, das muss der Kalender anzeigen können. Und eine normale Listenansicht, wo ich nach Wichtigkeit filtern kann, wäre auch cool.

Dann brauche ich einen Bereich für meine Noten. Das soll eine schöne Tabelle sein, wo alle meine Kurse mit den Noten und ECTS-Punkten drinstehen, am besten nach Semestern sortiert. Ich will Noten hinzufügen, bearbeiten und löschen können. Weil es nervt, alles einzutippen, wäre eine Funktion genial, wo ich eine CSV-Datei mit meinen Noten hochlade und die App schlau genug ist, zu erkennen, was in welcher Spalte steht. Der Notenschnitt muss sich dann automatisch anhand der ECTS-Gewichtung ausrechnen. Und die App muss aufpassen, dass ich nur Noten für Kurse eintragen kann, die auch wirklich zu meinem Studiengang gehören.

Für den Stundenplan stelle ich mir ein Raster vor: 5 Wochentage (Montag bis Freitag) und 6 feste Zeitblöcke am Tag. Die Zeiten an unserer Uni sind immer: 8:00-9:30, 9:45-11:15, 11:30-13:00, 14:00-15:30, 15:45-17:15 und 17:30-19:00. Man soll über ein Dropdown-Menü fertige Pläne von der Uni (die irgendwie aus den Daten der Uni-Website gezogen wurden) durchsuchen und auswählen können. Mit einem einfachen Plus- oder Minus-Button will ich dann Kurse zu meinem persönlichen Plan hinzufügen oder entfernen können. Man soll optisch sofort den Unterschied sehen zwischen den vorgeschlagenen Uni-Kursen und meinen selbst hinzugefügten. Und ein Button, um den ganzen Plan auf einmal zu löschen (nachdem man nochmal kurz gefragt wurde, ob man sich sicher ist), wäre top.

Ach ja, das Ganze muss natürlich sicher sein! Man muss sich registrieren und einloggen können, damit jeder nur seine eigenen Noten und Termine sieht. Und einen Ausloggen-Button brauche ich natürlich auch.

Ich habe leider überhaupt keine Ahnung von Programmieren oder wie man sowas baut, aber ich hoffe, du kannst mir daraus eine schicke, einfach zu bedienende App machen! Danke schonmal!

A.2 Prompt 2: Studiengruppen und Zusammenarbeit

Mir ist gerade noch etwas eingefallen: Ich würde die App gerne auch mit anderen aus meinem Semester benutzen können.

Vielleicht könnte man sogenannte Studiengruppen anlegen. Zum Beispiel für ein bestimmtes Modul, eine Lerngruppe oder die Bachelorarbeit. Eine Gruppe sollte einen Namen bekommen und man sollte andere Leute über einen Einladungslink oder per E-Mail hinzufügen können.

In einer Studiengruppe wäre es praktisch, wenn man gemeinsame Termine und Aufgaben verwalten könnte. Zum Beispiel:

Prüfungstermine
Abgabefristen
Lerntreffen
gemeinsame To-dos
wichtige Links oder Notizen
Bei Aufgaben sollte man ungefähr sehen können, worum es geht, wann sie fällig sind und wer sich darum kümmert. Man sollte Aufgaben auch als offen oder erledigt markieren können.

Zu gemeinsamen Terminen und Aufgaben sollten die Mitglieder Kommentare schreiben können. Außerdem wäre es hilfreich, wenn man sehen kann, wer etwas erstellt oder zuletzt geändert hat.

Die gemeinsamen Inhalte sollen nur für die Mitglieder der jeweiligen Gruppe sichtbar sein. Meine persönlichen Noten und privaten Termine sollen nicht automatisch mit anderen geteilt werden. Wenn ich etwas teilen möchte, sollte ich das selbst auswählen können.

Wenn jemand aus der Gruppe einen Termin oder eine Aufgabe ändert, sollten die anderen Mitglieder die Änderung sehen, wenn sie die App öffnen. Ich möchte die App außerdem mit meinem Benutzerkonto auch an einem anderen Gerät verwenden können, ohne dass meine Gruppen und Einträge dort fehlen.

Die Studiengruppen sollten sich möglichst einfach bedienen lassen und sich optisch gut in die bestehende Kalender- und Aufgabenansicht einfügen. Es wäre schön, wenn persönliche und gemeinsame Termine klar voneinander zu unterscheiden wären.